



《工程智能基础》

大模型与智能体

主讲人：杨濛、金仲明

同济大学 机械工程与机器人学院



目录

CONTENT

1

大语言模型基础

2

Qwen的应用

3

智能建造Agent

4

虚拟仿真



大语言模型是什么？

"大"

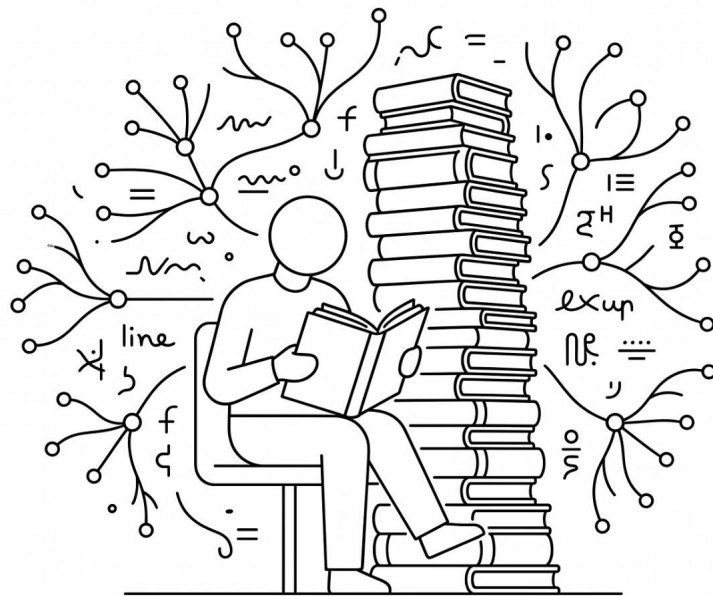
参数量巨大，从几十亿到几千亿不等

"语言"

处理的是人类语言——文字、词汇、句子

"模型"

一个经过大规模训练的数学函数



- ① 把它想象成：一个读过几乎所有书、所有网页、所有论文的人。你说上半句，它能接下半句。它不是“理解”语言，而是学会了语言的**统计规律**——什么词最可能出现在什么词后面。



Next Token Prediction

核心思想：给前文，预测下一个词

LLM 的训练目标极其简单，却威力无穷：给定前面所有的词，预测下一个词的概率分布。

数学表达

$$P(x_t \mid x_1, x_2, \dots, x_{t-1})$$

即：在已知 x_1 到 x_{t-1} 的条件下，第 t 个词的概率

预测示例

输入	"今天天气真"
"好"	概率 60%
"不错"	概率 25%
"热"	概率 10%
其他词	概率 5%

完形填空类比

| "春眠不觉晓，处处闻_____"

LLM 做的就是这件事，只不过规模大了亿万倍——
训练时见过数万亿个 Token，将所有语言规律压缩进
参数矩阵。



为什么 ChatGPT 只有 Decoder

生成任务的本质就是"往下写"——ChatGPT 以及几乎所有现代 LLM 都只用 Decoder

1 任务匹配

聊天、写文章、写代码——都是"给上文，续写下文"，正是 Decoder 擅长的

2 Next Token Prediction 天然适配

单向注意力机制，正好对应"只看前面、预测下一个"

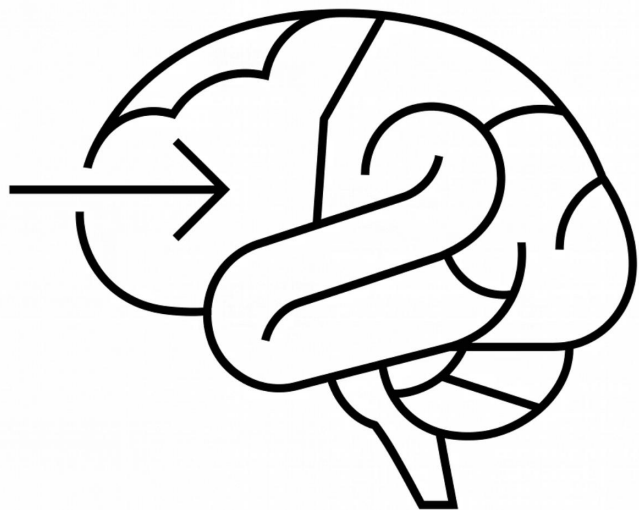
3 简洁统一

一个架构搞定所有生成任务，无需 Encoder + Decoder 两套系统

4 规模效应

参数全部集中在一个模块，扩大规模时效率更高

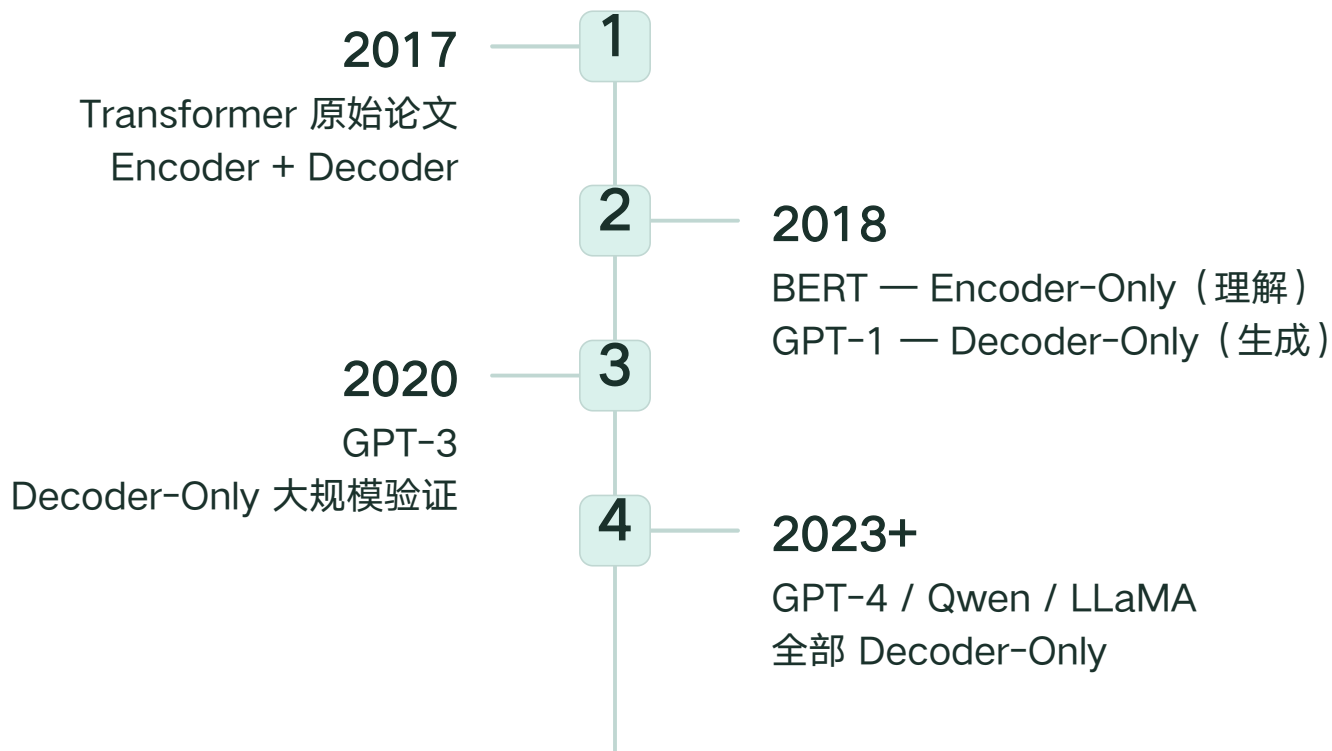
❑ 最早的 Transformer (2017) 是 Encoder + Decoder 的。后来人们发现：如果目标是生成文本，只留 Decoder 就够了，而且更好。





Decoder-Only 架构演进

为什么现代 LLM 都选这个



统一

一个架构处理所有任务：问答、翻译、写代码、聊天

简洁

架构简单，工程实现更方便

规模友好

参数集中，扩大规模时性能提升更明显



Transformer 全景

Transformer 是 2017 年 Google 提出的架构，是几乎所有大语言模型的基础。



核心模块说明

Token 嵌入 + 位置编码

把文字转成数字，并告知位置

Self-Attention

让每个词"看到"前面所有词

Feed-Forward

对每个词做进一步处理

Decoder Block 重复堆叠很多层
GPT-4 有上百层。

Transformer 全景

Decoder Block 内部结构（单层）

01

Masked Self-Attention

每个 token 只看自己和之前的 token

02

Add & LayerNorm

残差连接 + 层归一化：

$$x = \text{LayerNorm}(x + \text{Attention}(x))$$

03

Feed-Forward Network (FFN)

两层全连接：

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2,$$

维度 $d_{\text{ff}} \approx 4 \times d_{\text{model}}$

04

Add & LayerNorm

再次残差连接：

$$x = \text{LayerNorm}(x + \text{FFN}(x))$$

05

重复 N 次

GPT-3: N=96 层, Qwen3.5-72B: N=80 层

关键超参数对比

模型	d_{model}	层数	参数量
GPT-3	12288	96	175B
LLaMA-3-70B	8192	80	70B
Qwen3.5-72B	8192	80	72B
Qwen3.5-0.8B	1536	28	0.8B

Pre-Norm vs Post-Norm

Post-Norm（原始论文）

LayerNorm 在残差之后，训练不稳定，需要 warmup

Pre-Norm（现代主流）

LayerNorm 在残差之前，训练更稳定，Qwen/LLaMA 均采用



Transformer 的广泛应用

Transformer 不只属于大语言模型——它已渗透到深度学习的各个领域。

计算机视觉

ViT (Vision Transformer) —— 图像分类

DETR —— 端到端目标检测

蛋白质预测

AlphaFold —— 蛋白质结构预测，改变生命科学研究

语音识别

Whisper —— 语音转文字，支持多语言，实时转录

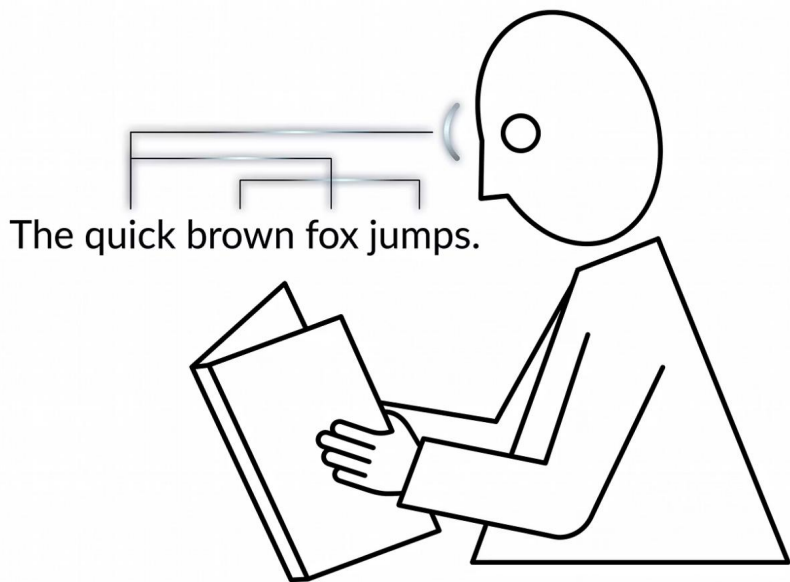
自然语言

GPT / Qwen / LLaMA —— 大语言模型，文本生成与对话



Self-Attention（直觉）

阅读时，眼睛会回看重要的词



直觉理解

当你读到"小明把球踢给了他"，你的眼睛会自动回看——"他"指的是谁？

模型也是这样：每个词会"回头看"前面所有的词，找到跟自己最相关的。

① "小明 把 球 踢给了 他"

↑ _____ ↑
"他"注意到"小明"最重要

- 注意力越强 → 连线越粗 → 关系越紧密
- 这是模型理解上下文的方式
- Attention 让模型知道哪些词跟当前词最相关



Self-Attention 公式

数学表达

Self-Attention 的核心公式将直觉转化为精确的线性代数操作：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其中 Q、K、V 来自输入的线性变换：

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

符号	全称	直觉含义
X	输入序列嵌入矩阵	每个词的初始向量表示
Q	Query (查询)	当前词想要"查询"什么信息
K	Key (键)	每个词提供的"索引标签"
V	Value (值)	每个词实际携带的信息内容
$\sqrt{d_k}$	缩放因子	防止点积过大

01

计算相关度分数

Q 和 K 的点积 QK^T ,
得到每对词之间的相似度矩阵

02

缩放 + Softmax

将分数归一化为概率分布
(概率之和等于1)

03

加权提取信息

概率矩阵乘以 V, 按相关度对各词的信息进行加权求和, 输出每个位置的新表示



位置编码（RoPE）

给每个词贴上座位号

问题

⚠ Transformer 本身不知道词的顺序。
"猫追狗" 和 "狗追猫"——
如果没有位置信息，对模型来说是一样的

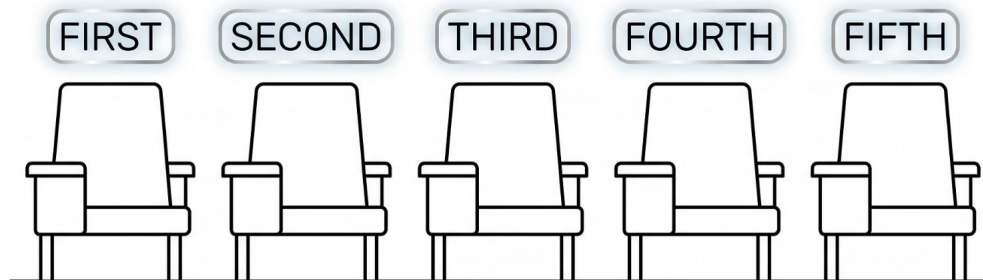
解决方案：位置编码

早期方法

固定的数学公式（正弦/余弦）

现代方法：RoPE

旋转位置编码——通过旋转向量来编码位置，
泛化能力更强



❑ 不需要知道 RoPE 的数学细节，只需要记住：它让模型知道每个词在句子中的位置



位置编码（RoPE）

给每个词贴上"座位号"

为什么需要位置编码？

Transformer 的 Self-Attention 本质上是"集合操作"——对一组向量做加权求和，本身**不感知顺序**。"猫追狗"和"狗追猫"在没有位置信息时是等价的。因此必须显式地将位置信息注入模型。

RoPE 核心公式

$$f(x_m, m) = x_m e^{im\theta}$$

将位置 m 编码为向量的**旋转角度**，不同位置对应不同的旋转，位置信息被"烙印"在向量方向中。

RoPE 的关键性质

相对位置感知

两个位置向量的内积只取决于**相对距离**，而非绝对位置，更符合语言的实际结构

长序列外推

天然支持外推到训练时未见过的更长序列

被广泛采用

GPT-4、Qwen、LLaMA 等现代主流 LLM 均使用 RoPE，已成为行业标准



Dense vs MoE

全员上阵 vs 专家分工

特性

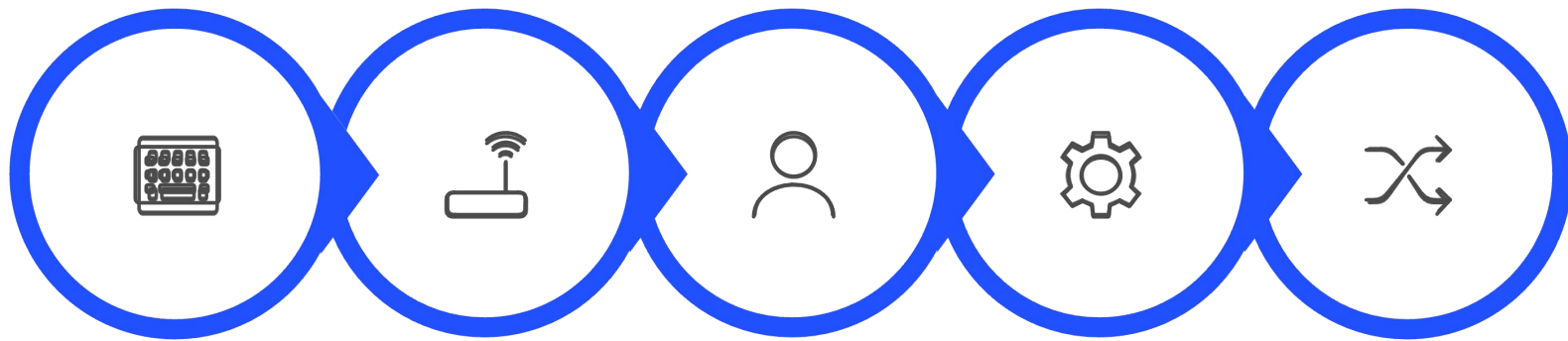
计算方式
参数利用
类比
代表模型

Dense（稠密模型）

每个 Token 经过**所有参数**
100% 参数参与每次推理
全公司所有人处理每件事
Qwen3.5-7B、GPT-4 早期版

MoE（混合专家模型）

每个 Token 只经过**被选中的专家**
通常仅 4%~15% 参数激活
不同部门各司其职，按需调用
Qwen3.5-397B-A17B、DeepSeek-V3



输入Token

路由网络

选择Top-K专家

专家并行计算

加权求和→输出

路由网络为每个 Token 计算对所有专家的亲和度分数，选取得分最高的 Top-K 个专家（通常 K=2 或 8），只激活这些专家的参数参与计算，其余专家“休眠”。这使得模型可以拥有极大的总参数量，却保持很低的推理计算量。



MoE 实例：Qwen3.5

397B 参数，只用 17B 干活

397B

总参数量

3970 亿参数，庞大的知识储量

17B

每次激活

推理时仅激活 170 亿参数

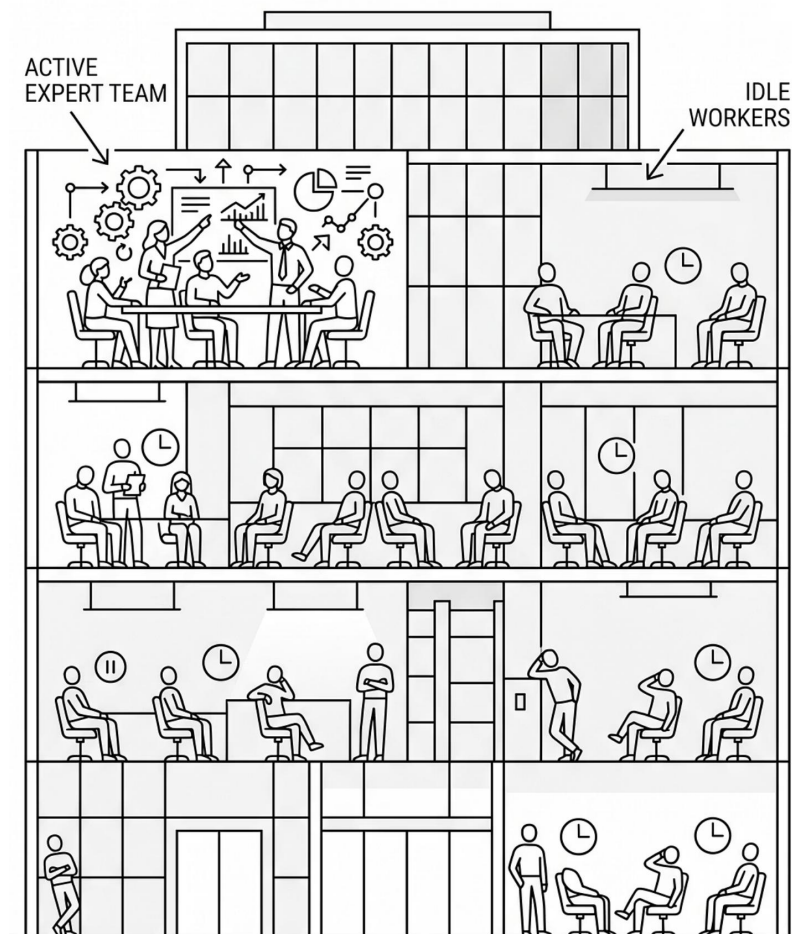
~4%

激活比例

相当于 397 个员工，每次只需 17 人上班

- ☑ 这意味着：模型"知识量"等同于 397B 大模型，但推理速度和成本接近 17B 小模型——又大又快又省钱。

LARGE COMPANY WITH INEFFICIENT
WORKFORCE DISTRIBUTION





训练三阶段总览

从"白纸"到"好助手"

一个能与你流畅对话的大模型，并非一步训练出来的。它经历了三个阶段，每个阶段解决一个不同的问题：会说话、会问答、说得好。

1

① 预训练 (Pretraining)

数据：数万亿 Token 的互联网文本

目标：学会语言规律，能"续写"

结果：会说话，但不会回答问题

2

② 监督微调 (SFT)

数据：几万条人工标注的问答对

目标：学会问答格式和对话风格

结果：能回答问题，但质量不稳定

3

③ RLHF

数据：人类对两个回答进行偏好比较

目标：学会人类偏好，输出更好

结果：回答清晰有条理，符合期望



训练三阶段总览

从"会说话"到"说人话"——大语言模型的训练分三个阶段逐步优化。

1

阶段一：预训练

Pretraining

学会语言规律

→ "能说话"

2

阶段二：有监督微调

SFT

学会回答问题

→ "说对话"

3

阶段三：人类反馈强化学习

RLHF

学会回答得让人满意

→ "说好话"

一、预训练（Pretraining）



海量阅读，学会语感

数据来源

互联网网页、书籍、论文、代码……
万亿级 Token

训练方法

Next Token Prediction——不断预测下一个词

数据规模

Qwen3.5 用了超过 36 万亿 Token

① 类比：一个小孩从出生开始大量阅读图书馆所有的书。他不一定“理解”每本书，但学会了语言的规律。

能生成流畅文本

不会好好回答问题



预训练 (Pretraining)

海量文本，学习语言规律

训练目标：最小化交叉熵损失

$$L = - \sum_{t=1}^T \log P(x_t \mid x_1, \dots, x_{t-1})$$

对序列中每个位置，模型预测下一个 Token 的概率分布，损失函数惩罚与真实下一词的偏差。训练目标极简，但规模带来了惊人的能力涌现。

训练规模

- **数据量**：数万亿 Token（相当于几百万本书）
- **计算量**：数千个 GPU 训练数周甚至数月
- **成本**：顶级模型预训练成本超过 1 亿美元

预训练后的模型特点

能做到

给一段文字续写；对文章风格进行模仿；完成语言规律的“统计压缩”

做不到

回答“你好，请问...”这样的问题；遵循指令；拒绝有害内容

- ❑ 预训练模型像一个读了所有书的“野生天才”——知识渊博，但不懂礼貌，也不知道怎么被使用。这就是为什么需要后续两个阶段。



二、有监督微调（SFT）

老师批改作业，教你标准答案

数据

人工标注的高质量问答对（问题 → 标准回答）

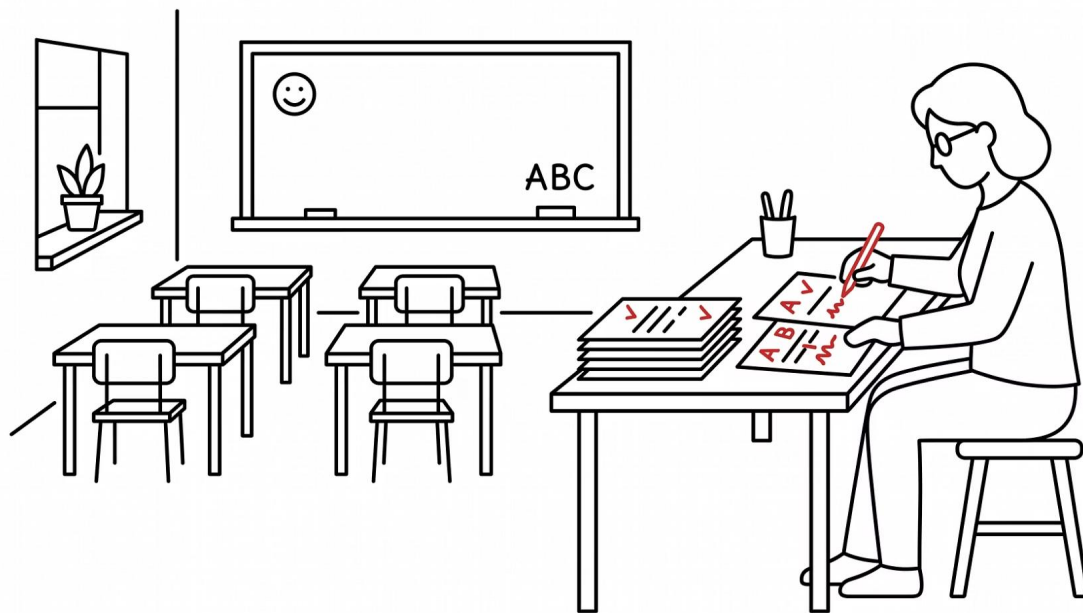
方法

让模型学习“看到这种问题，应该这样回答”

① 类比：小孩读了很多书之后，老师开始出题并给标准答案，让他学会“怎么正确回答问题”。

能针对问题给出合理回答

风格可能生硬、不自然





有监督微调（SFT）

用人工标注的问答对训练

训练数据格式

[System] 你是一个有帮助的助手。
[User] 什么是混凝土的养护？
[Assistant] 混凝土养护是指在混凝土浇筑后，通过保持适当温度和湿度，促进水泥水化反应完全进行的技术措施...

SFT 数据由专业标注人员编写，覆盖各类问答场景：知识问答、指令执行、角色扮演、代码生成、安全拒绝等。

SFT 的关键特点

数量少，质量高

通常只需几万~几十万条高质量样本，远少于预训练数据量。数据质量比数量更重要。

学会"问答格式"

SFT 后的模型理解了"用户问，助手答"的对话结构，能够识别指令并按格式输出。

局限性

SFT 依赖标注者的主观判断，不同标注者标准不一，导致输出质量不稳定，需要 RLHF 进一步对齐。

三、人类反馈强化学习（RLHF）



不只要答对，还要答得让人舒服

数据

人类对模型多个回答的偏好排序（A 比 B 好）

方法

训练"奖励模型"打分，再用强化学习优化

① 类比：老师不再给标准答案，而是比较你写的几篇作文，告诉你"这篇比那篇好"。你慢慢学会什么样的回答更受欢迎。

✓ 回答自然、有帮助、安全——这就是你用的 ChatGPT / Qwen 的最终形态。



RLHF 概述

让输出更符合人类偏好

RLHF (Reinforcement Learning from Human Feedback, 基于人类反馈的强化学习) 让模型从"能回答"升级为"回答得好"

收集偏好

01

收集偏好数据
对同一个问题，生成两个不同回答 A 和 B。人类标注者选择哪个更好。收集大量这样的比较对，形成偏好数据集。

训练奖励

02

训练奖励模型
用偏好数据训练一个独立的"评分模型" (Reward Model)，输入一个回答，输出一个质量分数。这个模型学习了人类的评价标准。

PPO优化

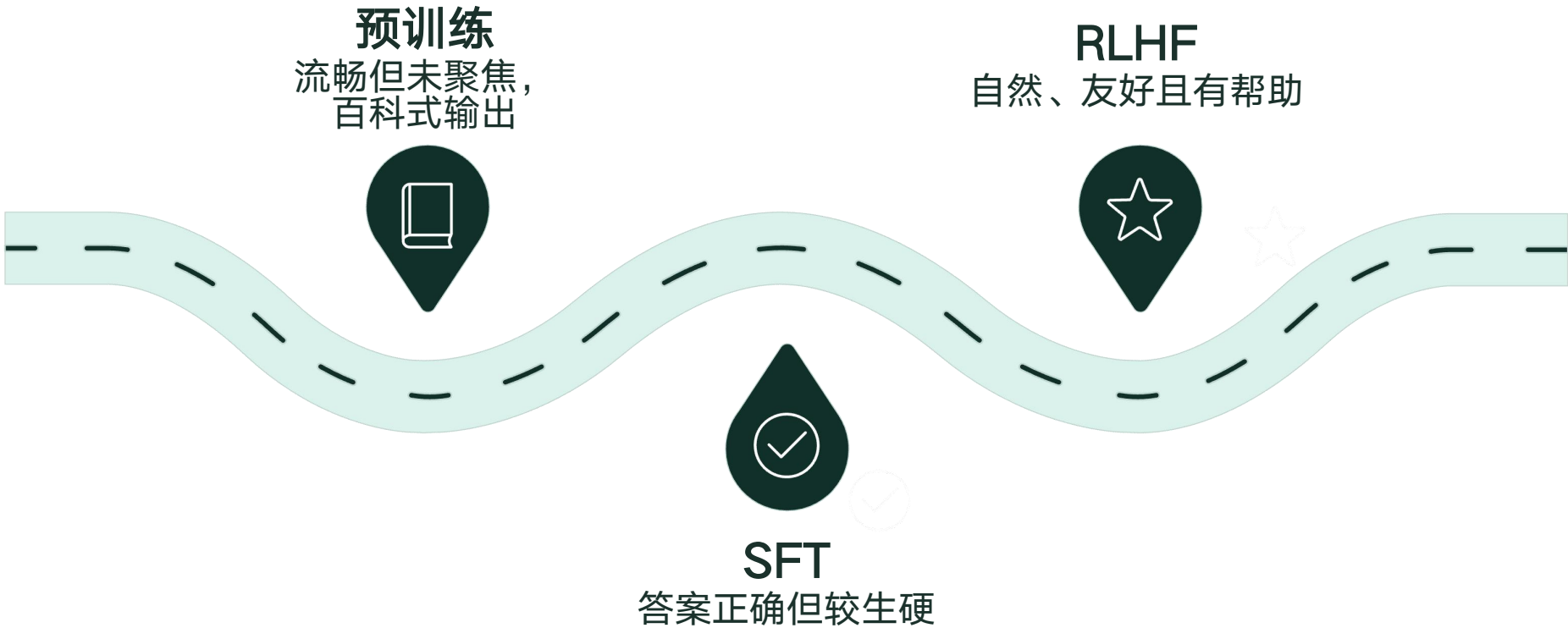
03

PPO 优化 LLM
用强化学习算法 (PPO) 训练 LLM，以奖励模型的高分为目标，调整 LLM 的参数，使其生成人类偏好的回答。



小结：三阶段效果对比

阶段	用户输入	模型表现
预训练	"中国的首都是"	"北京市，位于华北平原北部，东经116°..."（百科式输出，停不下来）
SFT	"中国的首都是哪里？"	"中国的首都是北京。"（能回答，但比较生硬）
RLHF	"中国的首都是哪里？"	"中国的首都是北京，它是中国的政治中心。还有什么想了解的吗？"





推理流程（Inference）

□ 注意：这里的"推理"是 Inference（使用模型生成输出），不是 Reasoning（逻辑推理能力）。

输入文本

分词编码

自回归生成

输出文本

整个过程就是不断重复 Next Token Prediction，直到模型输出结束标记。每次预测一个 Token，将结果拼回输入，再预测下一个。

Temperature 和 Top-p

控制模型的"创造力"

推荐设置

场景

写代码、回答事实

写故事、头脑风暴

推荐设置

Temperature 低, Top-p 低

Temperature 高, Top-p 高

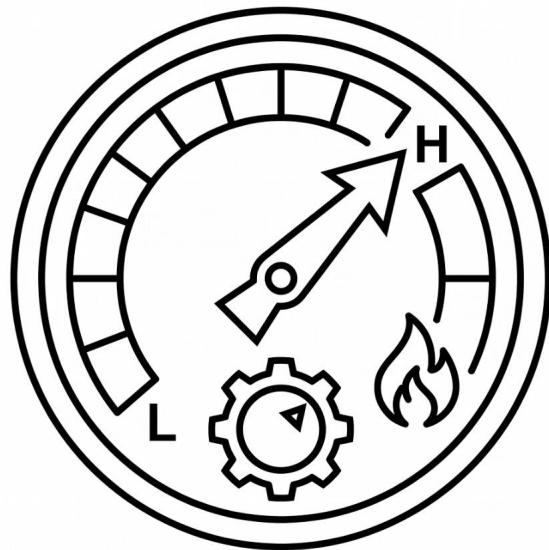
Temperature (温度)

考试时: 低温 = 只写最有把握的答案; 高温 = 敢猜、敢冒险

Top-p (核采样)

只从累计概率达到 p 的最可能词里选。

Top-p = 0.9 → 从概率最高的那些词 (合计占 90%) 里随机选一个。



低温 (如 0.1)

非常确定, 几乎总选概率最高的词 → 输出稳定、重复

高温 (如 1.5)

更随机, 会选概率较低的词 → 输出多样、有创意



Temperature 和 Top-p

控制输出的"创造力"

Temperature 公式

$$P_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

Temperature T 调节概率分布的"平坦程度":

- $T \rightarrow 0$: 概率集中在最高分 Token, 输出极度确定, 几乎没有随机性
- $T = 1$: 保持原始分布, 正常随机性
- $T > 1$: 分布趋于均匀, 低概率词也有机会被选中, 输出更随机创意

使用场景指南

场景	Temperature	效果
写代码	0.1~0.3	确定、保守, 减少语法错误
信息问答	0.3~0.6	准确, 偶有表达变化
日常对话	0.7~0.9	自然流畅
创意写作	1.0~1.5	随机、有创意, 可能出现惊喜

Top-p (核采样)

仅从累积概率达到 p 的最小词集合中采样。例如 top-p=0.9, 表示只考虑累积概率前 90% 的词汇。与 Temperature 配合使用, 过滤掉极低概率的"奇怪词"。



Inference vs Reasoning

两个"推理", 含义完全不同

	Inference (推理/生成)	Reasoning (推理/思考)
含义	模型生成输出的过程	模型进行逻辑推理的能力
类比	嘴巴在说话	脑子在思考
所有 LLM 都有?	是	不一定, 需要专门训练



普通 LLM

看到问题 → 直接回答 (可能答错复杂问题)



Reasoning LLM

看到问题 → 先想一想 → 再回答 (复杂问题表现更好)



思考链 (Chain of Thought)

让模型把推理步骤写出来

普通回答

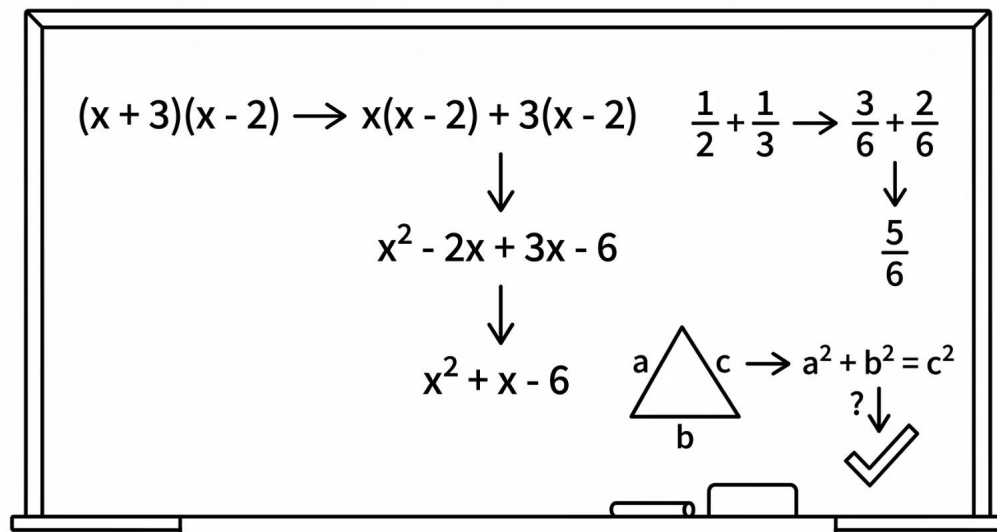
⚠ 问：进水管 3 吨/时，出水管 1 吨/时，5 小时后水池有多少吨？
答：10 吨
(但你不知道它怎么算的，也可能蒙对的)

思考链回答

✅ 问：(同上)
思考：净进水 = $3 - 1 = 2$ 吨/时，5 小时 = $2 \times 5 = 10$ 吨
答：10 吨
(推理过程清晰，结果更可靠)

代表模型

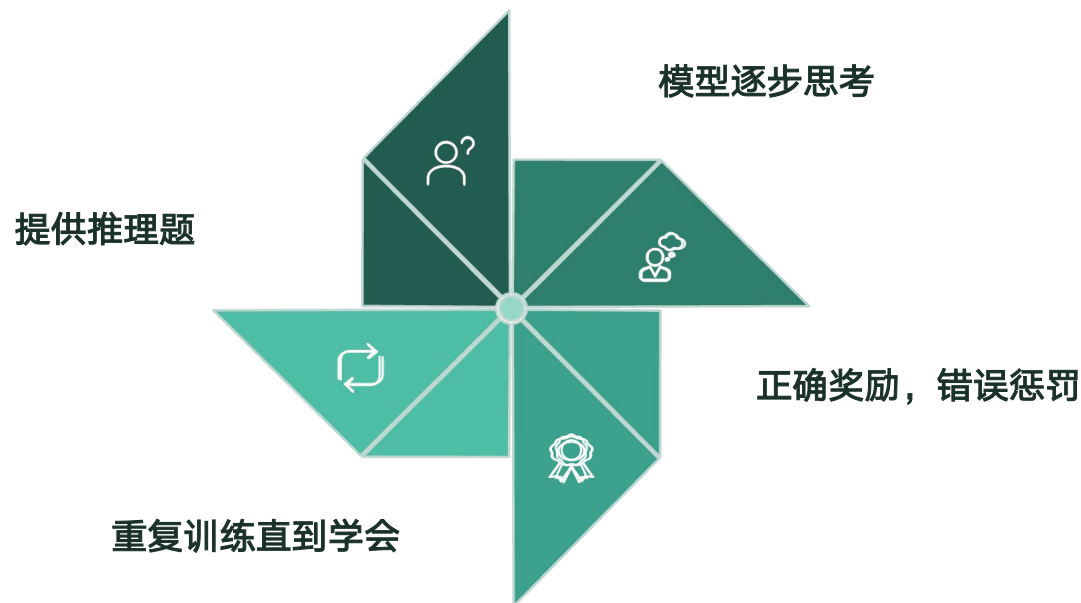
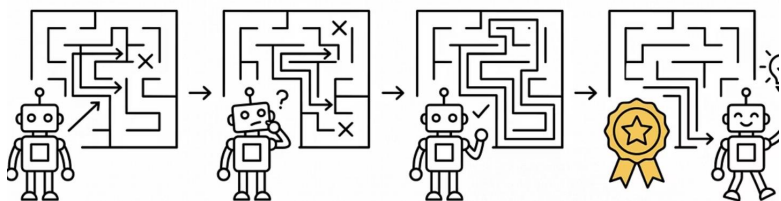
OpenAI o1 / o3 · DeepSeek-R1 · Qwen QwQ



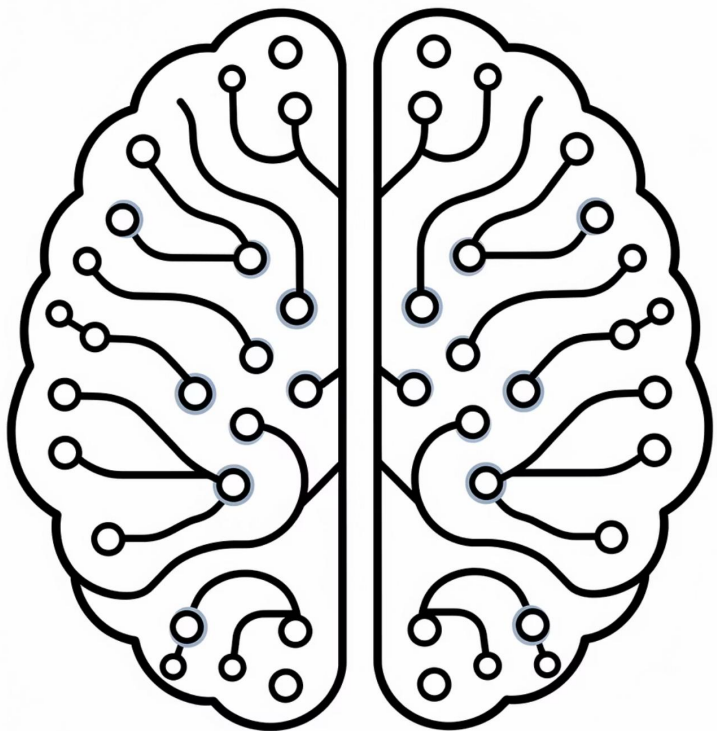


怎么做到的：RL 增强推理

用强化学习训练模型学会"思考"



DeepSeek-R1 的突破：纯 RL 训练就能让模型自发学会思考链，不需要人工标注思考过程。



RL 增强推理

用强化学习训练模型学会"思考"

继 CoT 提示工程之后，研究者发现可以用强化学习直接训练模型产生正确推理过程，而不只是在推理时加提示。

代表模型

QwQ (Qwen)、DeepSeek-R1、OpenAI o1/o3

训练策略

用 RL 奖励正确的推理过程，模型学会"先想再答"，自发产生中间步骤

推理时间扩展

推理 Token 越多 ("想得越久")，答案越准确



Transformer 是 LLM 的核心——也是连接语言、视觉、语音等所有现代 AI 的共同基础。



LLM 是什么

基于 Transformer 的大规模语言模型



核心原理

Next Token Prediction



网络结构

Attention · 位置编码 ·
Decoder-Only · MoE



训练过程

预训练 → SFT → RLHF



Reasoning

用 RL 增强推理能力，Chain of Thought

🕒 接下来：动手试试！ 进入实操环节。



目录

CONTENT

1

大语言模型基础

2

Qwen的应用

3

智能建造Agent

4

虚拟仿真

Qwen 家族概览

中国最强开源大模型之一——Qwen（通义千问）由阿里巴巴开发，全部 Apache 2.0 开源。

版本	发布时间	关键特性
Qwen3	2025 年	基础版本，多语言支持
Qwen3.5	2026 年 2 - 3 月	混合架构、MoE、256K 上下文
Qwen3.6	2026 年 4 月	最新版本，改进 Agentic Coding

📄 课程以 Qwen3.5 / 3.6 为例进行实践演示。

Qwen3.5 亮点

又大又快又开源

混合注意力机制

兼顾效率与效果，推理更高效

256K 上下文窗口

一次处理约 20 万字——相当于一本小说的长度

原生 FP8 训练

激活内存减少约 50%，训练更高效

MoE 架构

397B 总参数，仅 17B 活跃，性价比极高

250K 词表

支持 201 种语言和方言

全部开源

Apache 2.0 许可，任何人免费使用

Qwen3.5 模型规格

模型	类型	适用场景
Qwen3.5-0.8B	Dense	手机、嵌入式设备
Qwen3.5-2B	Dense	轻量级应用
Qwen3.5-4B	Dense	个人电脑
Qwen3.5-9B	Dense	工作站
Qwen3.5-27B	Dense	服务器
Qwen3.5-35B-A3B	MoE	高效推理
Qwen3.5-122B-A10B	MoE	高性能应用
Qwen3.5-397B-A17B	MoE	旗舰，最强性能

"B" = Billion (十亿) , "A" = Active (活跃参数)

阿里云 DashScope 可用模型

课上用哪个？——通过阿里云 DashScope API，可直接调用以下模型：

模型 ID	说明	推荐场景
qwen-max-latest	商业版，能力最强	复杂任务、高质量输出
qwen-plus-latest	商业版，性价比高	
qwen3.5-plus	Qwen3.5 系列	
qwq-plus	推理增强版	数学、逻辑推理

👉 课程实践中，我们使用的是 `qwen-max-latest`（每种模型有免费的 1M tokens 额度）



什么是 API

你不用进厨房，只需要告诉服务员你要什么



餐厅类比

1

你（程序）

点菜

2

服务员（API）

传递订单

3

厨房（大模型）

做菜，返回结果

- ① 你不需要自己训练模型（不用进厨房）
只需要发送请求（点菜）
API 会返回结果（端菜）
阿里云 DashScope 就是这个“餐厅”
Qwen 模型就是“厨师”

注册账号

- 复制网址：
<https://www.aliyun.com/product/tongyi>，
打开通义大模型网页，点击右上角注册。
- 根据提示词依次填写“手机号”和“校验码”，还需要勾选同意“我已阅读并同意”。



2

注册

阿里云APP/支付宝/钉钉



其他方式



[前往登录 >](#)

+86 请输入

验证码 请输入

[获取验证码](#)

☒ 信息补充 (可跳过)

账号名 请输入

密码 请输入

☐ 我已阅读并同意 [用户协议](#)、[隐私政策](#)、[产品服务协议](#)

立即注册

快速实名认证

- 提交完成后，表明账号已注册成功，然后会自动弹出快速实名认证，选择“个人认证”。
- 选择支付宝扫码快速认证之后，弹出“认证成功”界面，点击“返回”。

3

推荐您快速实名认证

个人认证

个人支付宝授权验证后即可立即完成认证
认证后可以享受 150 余款云产品的免费使用
二维码在 2026-02-25 11:54:06 后失效



切换为企业认证

企业法定代表人扫脸验证后即可立即完成认证

4

✓ 恭喜！账号注册认证成功

您可以前往 [账号中心](#) 修改登录名、设置密码、绑定邮箱、修改手机号等

账号名

绑定手机

通行密钥 [前往设置](#)

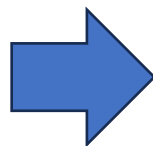
认证主体

证件号码

返回

部署在线大模型

- 完成之后进入网页点击“免费领取700万tokens”，获取免费的大模型使用额度。
- 点击“开始体验”，进入大模型界面后，同意开通百炼大模型服务，完成大模型部署。



测试在线大模型

阿里云百炼

模型广场

模型体验

1.在百炼大模型中选择文本模型

文本模型

语音模型

视觉模型

全模态模型

模型训练

欢迎使用千问大模型

开始体验 **Qwen3.5-Plus**

2.选择千问3.5plus模型

你好

+ 深度思考 搜索

3.输入提示词后模型回答，测试成功

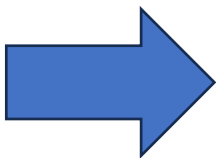
你好

你好！有什么我可以帮你的吗？



API密钥获取

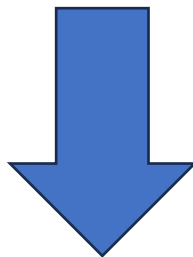
1. 选择大模型之后，点击右下角选择“密钥管理”。
2. 在密钥管理栏上选择“API-Key”，并点击网页右上角的“创建API-Key”。
3. 选择归属账号，在归属业务空间选择“默认业务空间”，描述选项可进行选填，然后点击“确定”，即可创建API-Key。
4. API-Key 为程序调用的密钥，点击查看后复制（需要记住本次生成的API-Key,建议将其保存在记事本）。





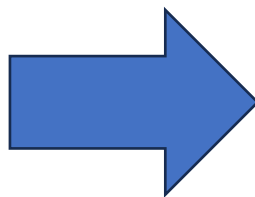
API密钥获取

1. 选择大模型之后，点击右下角选择“密钥管理”。
2. 在密钥管理栏上选择“API-Key”，并点击网页右上角的“创建API-Key”。
3. 选择归属账号，在归属业务空间选择“默认业务空间”，描述选项可进行选填，然后点击“确定”，即可创建API-Key。
4. API-Key 为程序调用的密钥，点击查看后复制（需要记住本次生成的API-Key,建议将其保存在记事本）。



API密钥获取

1. 选择大模型之后，点击右下角选择“密钥管理”。
2. 在密钥管理栏上选择“API-Key”，并点击网页右上角的“创建API-Key”。
3. 选择归属账号，在归属业务空间选择“默认业务空间”，描述选项可进行选填，然后点击“确定”，即可创建API-Key。
4. API-Key 为程序调用的密钥，点击查看后复制（需要记住本次生成的API-Key,建议将其保存在记事本）。



创建API Key

归属账号 *

用户名称	账号
☒ 13534*****22634	135345 [REDACTED]

4.勾选归属账号

归属业务空间 *

默认业务空间 5

描述

请输入描述

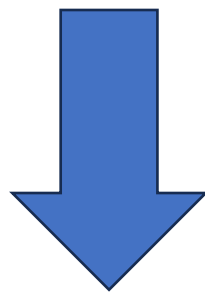
取消 确定



API密钥获取

1. 选择大模型之后，点击右下角选择“密钥管理”。
2. 在密钥管理栏上选择“API-Key”，并点击网页右上角的“创建API-Key”。
3. 选择归属账号，在归属业务空间选择“默认业务空间”，描述选项可进行选填，然后点击“确定”，即可创建API-Key。
4. API-Key 为程序调用的密钥，点击复制（需要记住本次生成的 API-Key,建议将其保存在记事本）。

ID	API Key	归属账号
3135703	6 sk-4****780e 	1353 



将API Key复制并记录至本地电脑的记事本便于后续使用多模态大模型功能时部署密钥



API密钥填入

1. 打开LanderPi红色终端，进入大模型目录
2. 打开API-Key配置文件
3. 在红框处的 ' ' 内填入API-Key

```
> cd /home/ubuntu/ros2_ws/src/large_models/large_models/large_models  
└─┬─ | └─ ~ / ros2_ws / s / large_models / large_models / large_models | 🐱 🐭 main !231 ?28
```



API密钥填入

1. 打开LanderPi红色终端，进入大模型目录
2. 打开API-Key配置文件
3. 在红框处的 ' ' 内填入API-Key

```
~ /ros2_ws/s/large_models/large_models/large_models  
└─ gedit config.py
```



API密钥填入

1. 打开LanderPi红色终端，进入大模型目录
2. 打开API-Key配置文件
3. 在红框处的 ' ' 内填入API-Key

```
6 ###Only in China###
7 # 阶跃星辰key
8 stepfun_api_key = ''
9 stepfun_base_url = 'https://api.stepfun.com/v1'
10 stepfun_llm_model = ''
11 #'step-1v-8k'/'step-10-vision-32k'/'step-1.5v-mini'
12 stepfun_vllm_model = 'step-10-vision-32k'
13
14 # 阿里云key
15 aliyun_api_key = ''
16 aliyun_base_url = 'https://dashscope.aliyuncs.com/compatible-mode/v1'
17 aliyun_llm_model = 'qwen-max-latest' #'qwen-turbo' #'qwen-max-latest'
18 aliyun_vllm_model = 'qwen-vl-max-latest'
19 aliyun_tts_model = 'sambert-zhinan-v1'
20 aliyun_asr_model = 'paraformer-realtime-v2'
21 aliyun_voice_model = ''
22 #####
23
```

API 调用代码

运行前先安装依赖：pip install openai

```
import sys
from openai import OpenAI
sys.stdout.reconfigure(encoding="utf-8")
client = OpenAI(
    api_key="sk-xxx", # 你的 API Key
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
)
response = client.chat.completions.create(
    model="qwen-plus-latest", # 使用的模型名称
    messages=[
        {"role": "system", "content": "你是一个建筑工程助手。"}, # 给 AI 设定身份
        {"role": "user", "content": "请用简单的语言解释什么是混凝土的养护？"} # 用户的问题
    ]
)
print("=" * 50)
print("AI 回答：")
print("=" * 50)
print(response.choices[0].message.content)
```

运行结果展示

⋮ powershell

```
PS F:\others\robot> py qwen_demo.py
```

=====
AI 回答：

=====
混凝土的养护，简单来说就是：****混凝土浇筑完成后，给它“喝水”、盖“被子”，让它慢慢、健康地变硬变结实的过程。****

就像刚出生的小宝宝需要好好照顾一样，刚浇好的混凝土虽然看起来是硬的，但其实内部的水泥还在和水发生化学反应（叫“水化反应”），这个反应需要****水分****和****合适的温度****才能顺利进行。如果太干、太冷或太热，混凝土就容易：

- 开裂（缺水干缩）
- 强度不够（反应没完成）
- 表面起粉、掉皮（表面失水太快）

✅ 所以养护就是做这几件事：

- ◆ ****保湿****：用洒水、盖湿麻布、塑料薄膜或喷养护剂等方式，防止水分过快蒸发；
- ◆ ****保温****（尤其冬天）：用草帘、保温毯等盖住，避免冻坏；
- ◆ ****保持时间****：一般至少要养 ****7天****，重要结构常养 ****14天或更久****。

💡 小比喻：养护就像给混凝土“坐月子”——不急着用，耐心照顾，它才能长得壮、耐久、不出问题！

需要我告诉你不同天气下怎么具体养护吗？ 😊

System Prompt 的作用

改变 system prompt → 模型行为完全不同

System Prompt	用户问题	模型回答风格
"你是一个建筑工程助手"	什么是混凝土养护？	专业、工程角度
"你是一个幼儿园老师"	什么是混凝土养护？	简单、生动、比喻多
"你是一个诗人"	什么是混凝土养护？	可能写成一首诗

-  System Prompt 就像给模型设定了一个"人设"，它会按照这个人设来回答所有问题。
这是工程实践中最重要的调参手段之一。



改一改

01

换个模型

把 model 换成 `qwq-plus`（推理增强版），看看有何不同

02

换个人设

把 system prompt 改成你喜欢的角色，观察回答风格变化

03

问专业问题

提一个专业相关的问题，看看大模型如何作答



这就是你第一次和大语言模型"对话"的完整过程！





实践零、学生编码练习：文字对话

任务说明

- 1 修改角色与风格
修改 system 内容，把模型变成不同角色
- 2 尝试不同问题
修改 user 内容，向模型提问不同问题
- 3 实现多轮对话
把模型的回答追加到 messages 列表，
再发送新问题，实现真正的上下文对话

每次调用时把完整的 messages 列表发给模型，模型才能“记住”之前的对话。上下文窗口有限，超长对话需要截断或摘要。

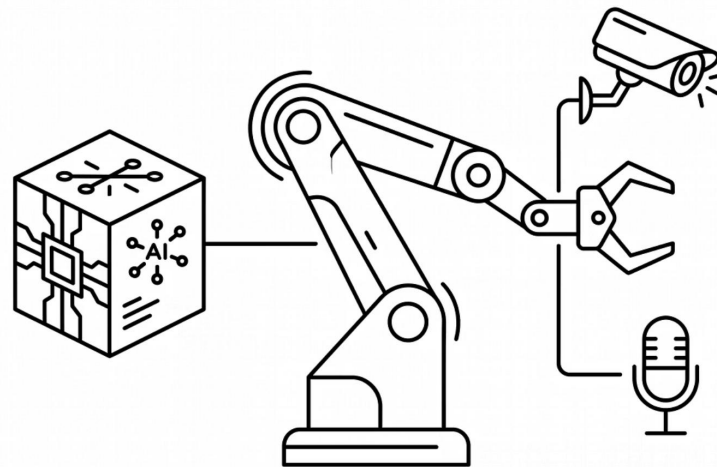
多轮对话框架

```
messages = [  
    {"role": "system",  
     "content": "你是一个建筑工程助手。"}  
]  
  
while True:  
    user_input = input("你: ")  
    messages.append({  
        "role": "user",  
        "content": user_input  
    })  
    response = client.chat.completions.create(  
        model="qwen-plus-latest",  
        messages=messages  
    )  
    reply = response.choices[0].message.content  
    print(f"AI: {reply}")  
    messages.append({  
        "role": "assistant",  
        "content": reply  
    })
```



API 只是起点

刚才我们用 API 和大模型"聊了一次天"。但真正的威力在于应用——让它调用工具、控制设备、做决策。



什么是 Tool Calling

让大模型不只"说", 还能"做"

从聊天到行动

如果我们告诉大模型"你有一些工具可以用", 它就能:

01

理解用户意图

分析用户指令, 判断需要执行什么操作

02

决定调用哪个工具

从可用工具列表中选择合适的工具

03

返回结构化 JSON

告诉程序该做什么, 而非自然语言回复

示例流程

- ① 用户: "帮我选一个黄色方块"
 - ↓ 大模型思考: 需要调用"选择方块"工具
 - ↓ 返回 JSON:

```
{"name": "select_block", "arguments": {"color": "yellow"}}
```
 - ↓ 程序解析 JSON → 执行对应操作

这就是 Tool Calling (工具调用), 也叫 Function Calling。



Qwen的应用：Tool Calling

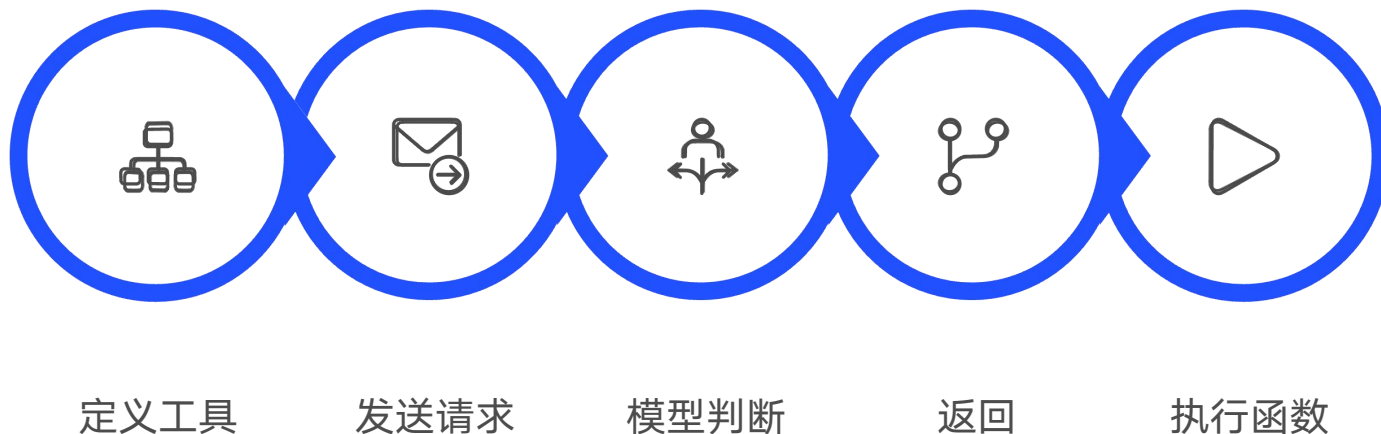


同济大学
TONGJI UNIVERSITY

Tool Calling 机制原理

让大模型"调用工具"

Tool Calling 让大模型能够主动调用外部函数、API、传感器，完成现实世界的任务。



工具定义

用 JSON Schema 描述工具的名称、参数类型、用途说明。这是大模型理解"我有什么工具"的唯一依据。

模型决策

大模型自主判断是否需要调用工具，选择哪个工具，填写哪些参数——这个决策完全由模型完成，无需硬编码。

结构化输出

模型返回标准 JSON 格式的调用请求，程序直接解析后执行，无歧义，易于自动化处理。



Tool Calling 示例代码

```
14 import sys
15 import json
16 from openai import OpenAI
17
18 sys.stdout.reconfigure(encoding="utf-8")
19
20
21 # =====
22 # 第 1 部分：定义"工具"——一个真正会被 Python 执行的函数
23 # =====
24 def select_block(color: str, reason: str) -> dict:
25     """模拟机器人抓取指定颜色方块的动作。"""
26     color_map = {
27         "yellow": "黄色方块",
28         "log": "橡木方块",
29         "cherry": "樱花木方块",
30     }
31     block_name = color_map.get(color, "未知方块")
32     print(f"\n🤖 [机器人执行] 正在抓取 {block_name} (颜色代号: {color})")
33     print(f"    选择理由: {reason}")
34     return {
35         "status": "success",
36         "picked_block": block_name,
37         "color": color,
38     }
39
40
41 # 把本地函数登记到一个表里，方便后面根据 AI 给的"函数名"找到对应函数
42 AVAILABLE_FUNCTIONS = {
43     "select_block": select_block,
44 }
```



Tool Calling 示例代码

```
47  # =====
48  # 第 2 部分：用 JSON Schema 告诉 AI 都有哪些工具可用
49  # =====
50  tools = [{
51      "type": "function",
52      "function": {
53          "name": "select_block",
54          "description": "选择指定颜色的方块进行抓取",
55          "parameters": {
56              "type": "object",
57              "properties": {
58                  "color": {
59                      "type": "string",
60                      "enum": ["yellow", "log", "cherry"],
61                      "description": "方块颜色: yellow=黄色, log=橡木, cherry=樱花木"
62                  },
63                  "reason": {
64                      "type": "string",
65                      "description": "选择这个颜色的原因"
66                  }
67              },
68              "required": ["color", "reason"]
69          }
70      }
71  }]
72
```

Tool Calling 示例代码

```
74  # =====
75  # 第 3 部分：创建客户端
76  # =====
77  client = OpenAI(
78      api_key="sk-xxx",
79      base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
80  )
81
82  messages = [
83      {"role": "system", "content": "你是一个建筑机器人助手，可以根据用户的需求挑选并抓取方块。"},
84      {"role": "user", "content": "我想搭一个温暖色调的底座，帮我选一个合适的方块。"}
85  ]
86
```


Tool Calling 示例代码

```
88 # =====
89 # 第 4 部分: 第一次调用 AI—AI 会决定"要不要调工具、调哪个工具"
90 # =====
91 print("=" * 60)
92 print("👤 用户提问: ", messages[-1]["content"])
93 print("=" * 60)
94
95 response = client.chat.completions.create(
96     model="qwen-plus-latest",
97     messages=messages,
98     tools=tools,          # 把工具清单交给 AI
99     tool_choice="auto",   # 让 AI 自己决定要不要用工具
100 )
101
102 ai_message = response.choices[0].message
103 print("\n🤖 AI 的第一次回复 (决策阶段): ")
104 print("    content :", ai_message.content)
105 print("    tool_calls:", ai_message.tool_calls)
106
```



Tool Calling 示例代码

```
108 # =====
109 # 第 5 部分：如果 AI 决定调用工具，就在本地执行这些工具
110 # =====
111 if ai_message.tool_calls:
112     messages.append(ai_message)
113
114     for tool_call in ai_message.tool_calls:
115         func_name = tool_call.function.name
116         func_args = json.loads(tool_call.function.arguments)
117
118         print(f"\n AI 决定调用工具: {func_name}")
119         print(f"    参数: {func_args}")
120
121         func = AVAILABLE_FUNCTIONS[func_name]
122         result = func(**func_args)
123
124         messages.append({
125             "role": "tool",
126             "tool_call_id": tool_call.id,
127             "content": json.dumps(result, ensure_ascii=False),
128         })
129
```

```
130 # =====
131 # 第 6 部分：把工具结果发回 AI，让它生成最终回答
132 # =====
133 final_response = client.chat.completions.create(
134     model="qwen-plus-latest",
135     messages=messages,
136 )
137 print("\n" + "=" * 60)
138 print("AI 最终回答: ")
139 print("=" * 60)
140 print(final_response.choices[0].message.content)
141
142 else:
143     print("\n(AI 没有调用工具，直接给出了回答)")
144     print(ai_message.content)
145
```

Tool Calling 解读

```
tools = [{  
    "name": "select_block",    # 工具名称  
    "description": "选择指定颜色...", # 告诉大模型这个工具干什么  
    "parameters": {          # 工具需要哪些参数  
        "color": {...},      # 参数1: 颜色 (要限定有什么值)  
        "reason": {...}      # 参数2: 原因 (让模型解释决策)  
    }  
}]
```

tools 参数

告诉大模型"你有哪些工具可以用"

自主决策

大模型根据问题, 自己决定要不要调用工具、调用哪个、传什么参数

结构化输出

返回的不是自然语言,
而是程序可直接解析的 JSON

运行结果展示

```
=====
👤 用户提问： 我想搭一个温暖色调的底座，帮我选一个合适的方块。
=====

💡 AI 的第一次回复（决策阶段）：
content :
tool_calls: [ChatCompletionMessageFunctionToolCall(id='call_0e491e3e76cc46a3ac4844', function=Function(arguments='{"color": "cherry", "reason": "樱花木颜色偏粉红，属于温暖色调，适合搭建温暖风格的底座"}', name='select_block'), type='function', index=0)]

AI 决定调用工具：select_block
参数：{'color': 'cherry', 'reason': '樱花木颜色偏粉红，属于温暖色调，适合搭建温暖风格的底座'}

🤖 [机器人执行] 正在抓取 樱花木方块（颜色代号：cherry）
选择理由：樱花木颜色偏粉红，属于温暖色调，适合搭建温暖风格的底座

=====
AI 最终回答：
=====
已成功为您选取 **樱花木方块** —— 柔和的粉红木质色调，自带温馨、柔和的暖意，纹理细腻，非常适合作为温暖风格底座的基础材料。需要我帮您规划底座尺寸（如 5x5 或 7x7）、添加边缘装饰（比如浅橡木围边），或搭配其他暖色方块（如蜂蜜块、橙色羊毛、砖块）来增强层次感吗？ 😊
```

大模型没有直接回答“我建议你用黄色”，而是返回了结构化 JSON，程序可以直接解析和执行。

程序解析 JSON

→ 调用机器人控制接口

→ 让机械臂去抓取黄色方块

这就是从“聊天”到“行动”的关键转变。

实践任务一：工具调用

通过API，让大模型分别调用：

①视觉识别 ②抓取 ③放置 ④移动 （可加入语音输入功能）

本次实践将完成 4 个独立的 ROS 2 节点，每个节点演示大模型对 1 个工具的调用。

4 个独立节点：

项目	节点入口	工具函数	功能
项目 1	<code>test_detect_tool</code>	<code>detect_blocks</code>	视觉识别桌面方块
项目 2	<code>test_grasp_tool</code>	<code>grasp_object</code>	控制机械臂抓取
项目 3	<code>test_place_tool</code>	<code>place_object</code>	控制机械臂放置
项目 4	<code>test_move_tool</code>	<code>move_robot</code>	控制底盘移动

4 个节点共用同一套 Tool Calling 框架，差异只在：

1. 工具定义（`xxx_tool` 字典）
2. 系统提示词（`xxx_prompt`）
3. 工具函数（`execute_xxx` 方法）



Qwen的应用：工具模板，节点结构模板



同济大学
TONGJI UNIVERSITY

```
TOOL = {
    "type": "function",
    "function": {
        "name": "tool_name",
        "description": "工具说明，告诉大模型这个工具做什么",
        "parameters": {
            "type": "object",
            "properties": {
                "param_name": {
                    "type": "number", # 或 "string"
                    "description": "参数说明",
                    # 可选: "enum": [...]
                }
            },
            "required": ["param_name"]
        }
    }
}
```

```
class TestXxxToolNode(Node):
    def __init__(self):
        rclpy.init()
        super().__init__('test_xxx_tool', ...)
        self._init_llm()           # 1. 初始化 LLM
        self._init_xxx()           # 2. 初始化硬件
        # 3. 订阅用户指令 Topic
        self.create_subscription(
            String, '~/user_query', self.user_query_callback, 1)
        # 4. 提供 fallback Service (不经大模型直接测硬件)
        self.create_service(
            Trigger, '~/xxx_test', self.xxx_test_callback, ...)

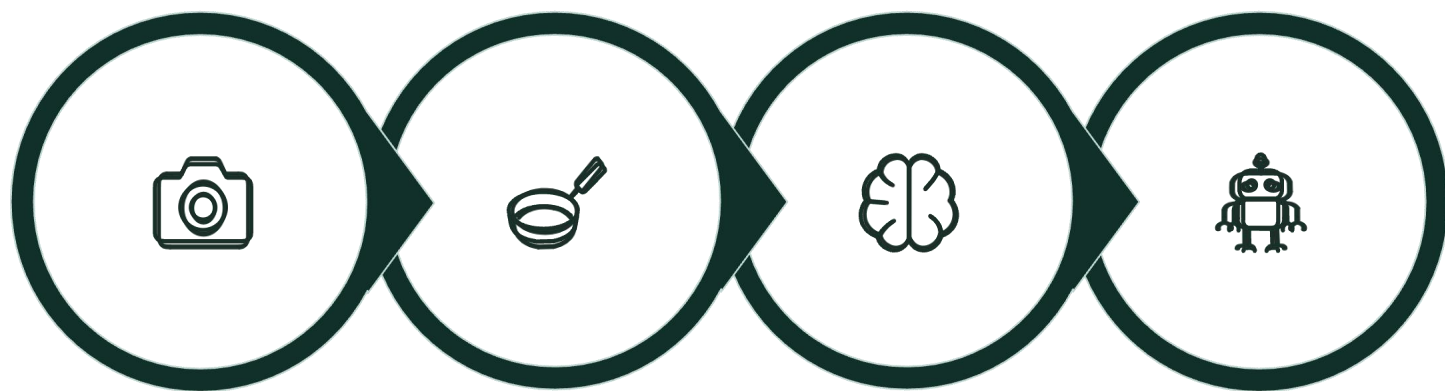
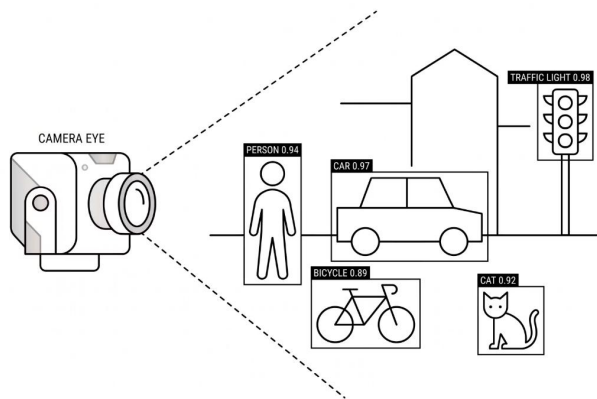
    def user_query_callback(self, msg):
        threading.Thread(
            target=self._run_tool_calling,
            args=(msg.data,), daemon=True).start()
```

工具一：视觉识别

【工具要求】

- 编写一个 ROS 2 Python 脚本 `test_detect_tool_node.py`，初始化并创建一个名为 `test_detect_tool` 的节点。
- 定义视觉工具 `DETECT_TOOL`：工具名为 `detect_blocks`，无需任何参数（`required: []`），用于触发深度相机拍照并识别方块。
- 编写系统提示词 `DETECT_ONLY_PROMPT`，告诉大模型该工具的作用、方块种类（`yellow / log / cherry`）、坐标系约定。
- 实现工具函数 `execute_detect_blocks()`：从同步话题取 `RGB + Depth` → `YOLOv11` 推理 → `NMS` → 像素坐标 + 深度 → 转换为机器人坐标系 3D 坐标 → 返回 JSON 字符串。
- 创建一个订阅者，订阅话题 `~/user_query`（`std_msgs/msg/String`，队列长度 1），在回调中起新线程执行完整的 5 阶段 Tool Calling 流程。
- 在主流程 `_run_tool_calling(query)` 中，依次调用 `client.llm_tools(...)` 与 `client.llm_tools_result(...)`，并按【阶段 1】~【阶段 5】格式打印日志。

让机器人看懂世界



拍照

YOLO检测

大模型理解

调用其他工具

① YOLO：实时目标检测模型，能识别图像中的物体和位置。
大模型不直接处理图像，而是调用 YOLO 小模型获取检测结果，再做出决策。

YOLOv11 视觉检测（让机器人看到方块）

YOLO 原理

You Only Look Once——一次前向传播就能检测图像中所有物体，速度快，适合实时应用。我们使用 YOLOv11，通过 OpenVINO 加速推理。

每个检测结果包含

类别

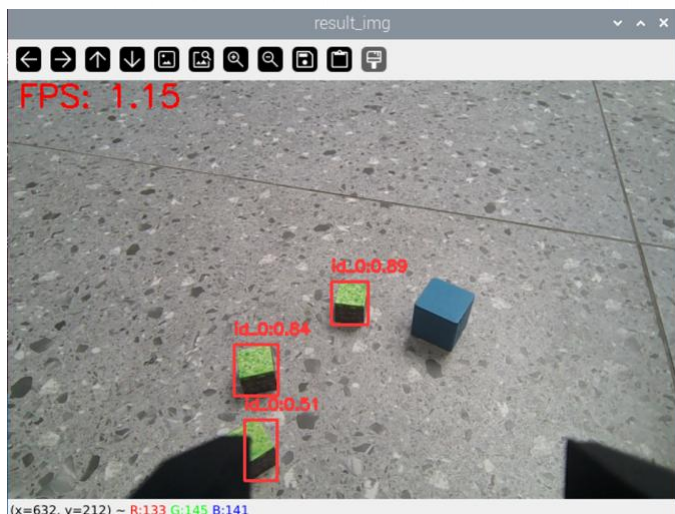
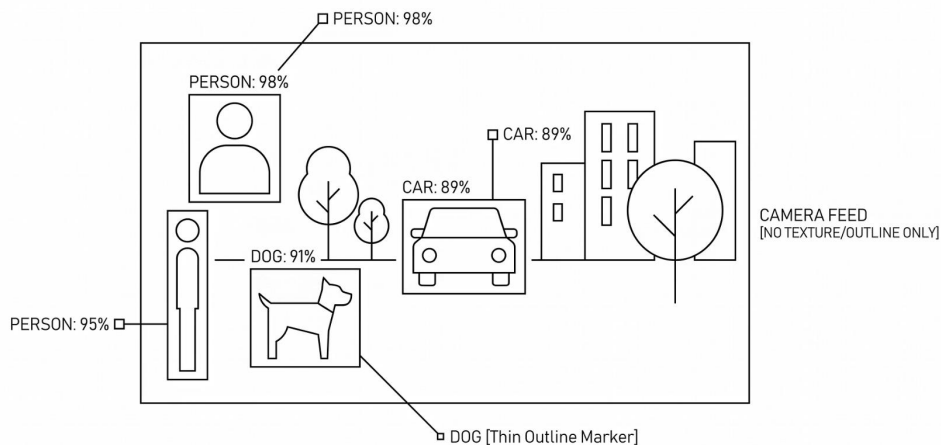
yellow / log / cherry（或者其他方块名称）

位置

像素坐标 (x, y)

置信度

检测可信程度 0 - 1



视觉工具第1部分：YOLOv11 检测

```
# 图像预处理：缩放到 640×640，归一化
blob = cv2.dnn.blobFromImage(
    input_image,
    scalefactor=1/255,
    size=(640, 640),
    swapRB=True
)

# OpenVINO 推理
outputs = self.ir.infer(blob)[self.output_node]

# NMS（非极大值抑制）：去除重叠的检测框
indices = cv2.dnn.NMSBoxes(boxes, scores, 0.25, 0.45)
```

blobFromImage

把图片转成模型需要的格式（归一化、尺寸统一）

infer

运行 YOLO 模型，得到所有检测结果

NMSBoxes

同一个方块可能被检测多次，NMS 只保留置信度最高的那个

视觉工具第2部分：深度相机 3D 坐标获取

从 2D 图像到 3D 坐标

为什么需要深度相机？

普通摄像头

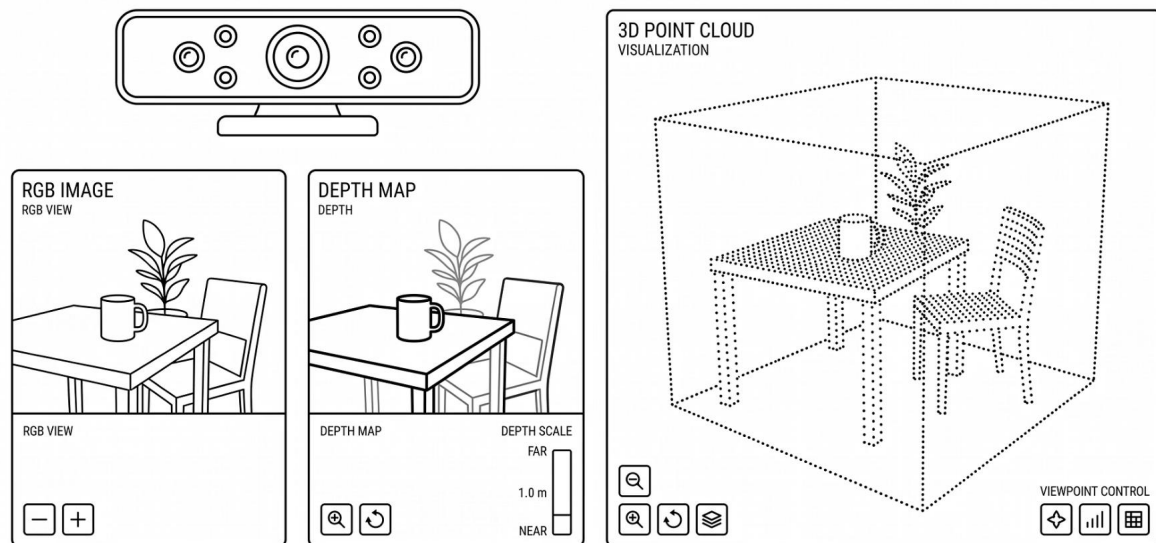
"方块在画面左上角" → (x, y) 只有二维位置，不知道距离

深度相机

"方块在画面左上角，距离 30cm" → (x, y, z) 三维位置，精确定位

✔ 有了 3D 坐标，机械臂才知道该伸到哪里去抓。

DEPTH CAMERA INTERFACE





视觉工具第3部分：坐标转换

像素坐标 → 相机坐标 → 机器人坐标

第一步：像素坐标 → 相机坐标系

$z_c = \text{depth_mm} / 1000.0$ # 深度（毫米→米）

$x_c = (\text{pixel_x} - \text{self.cx}) * z_c / \text{self.fx}$ # 水平偏移

$y_c = (\text{pixel_y} - \text{self.cy}) * z_c / \text{self.fy}$ # 垂直偏移

第二步：相机坐标 → 机器人坐标系（几何变换）

$px = (z_c * \cos(\theta)) - (y_c * \sin(\theta)) + \text{camera_offset_x}$

$py = -x_c$

$pz = \text{camera_height} - (y_c * \cos(\theta) + z_c * \sin(\theta))$

cx, cy, fx, fy

相机内参，出厂标定好的固有属性

θ （相机角度）

相机安装的俯仰角度

最终结果

方块在机器人坐标系中的精确位置 (px, py, pz)



Qwen的应用：工具一、视觉



同濟大學
TONGJI UNIVERSITY

```
DETECT_TOOL = {  
    "type": "function",  
    "function": {  
        "name": "detect_blocks",  
        "description": (  
            "使用机器人深度相机拍照，调用 YOLOv11 模型检测工作台上的方块。"  
            "返回每个方块的颜色类型 (yellow/log/cherry)、置信度和 "  
            "3D 坐标 (机器人坐标系，单位米)。该工具无需参数。"  
        ),  
        "parameters": {  
            "type": "object",  
            "properties": {},  
            "required": []  
        }  
    }  
}
```




```
DETECT_ONLY_PROMPT = """你是一个机器人视觉助手。你有一个工具 detect_blocks，  
可以使用机器人的深度相机拍照并识别工作台上的方块。
```

```
当用户询问关于方块的信息（位置、数量、种类等）时，请调用 detect_blocks  
工具获取检测结果，然后根据结果回答用户。
```

```
## 方块类型
```

- yellow: 黄色方块 (2cm × 2cm × 2cm)
- log: 橡木方块
- cherry: 樱花方块

```
## 坐标系（机器人底盘地面投影为原点）
```

- X轴：机器人前方为正（米）
- Y轴：机器人左侧为正（米）
- Z轴：向上为正（米，地面 ≈ 0 ）

```
## 回答要求
```

1. 列出每个方块的类型、置信度、坐标
2. 用自然语言总结检测结果
3. 如果没有检测到方块，说明可能的原因

```
"""
```




Qwen的应用：工具一、视觉



同濟大學
TONGJI UNIVERSITY

```
def execute_detect_blocks(self):
    # 1. 取最新同步帧 (RGB + Depth)
    with self.frame_lock:
        frame = self.latest_frame
    if frame is None:
        return json.dumps({"error": "相机未就绪"}, ensure_ascii=False)
    rgb_image, depth_image = frame

    # 2. YOLO 推理 (640×640 输入, 前向 + NMS)
    blob = cv2.dnn.blobFromImage(
        input_image, scalefactor=1/255, size=(640, 640), swapRB=True)
    outputs = self.ir.infer(blob)[self.output_node]
    indices = cv2.dnn.NMSBoxes(boxes, scores, 0.25, 0.45)

    # 3. 像素 + 深度 → 机器人坐标系
    detections = []
    for idx in indices:
        center_x, center_y = ... # 检测框中心像素
        depth_val = self._get_depth(depth_image, center_x, center_y)
        pos_3d = self._pixel_to_ground(center_x, center_y, depth_val)
        detections.append({
            'type': cls_name,
            'confidence': round(cls_conf, 3),
            'position': [round(p, 3) for p in pos_3d],
        })

    return json.dumps(detections, ensure_ascii=False, indent=2)
```



Qwen的应用：工具一、视觉



同濟大學
TONGJI UNIVERSITY

```
def _pixel_to_ground(self, px, py, depth_mm):
    """像素 + 深度 → 机器人坐标系（地面投影为原点）"""
    if depth_mm <= 0 or depth_mm > 10000:
        return None

    # 1. 像素 → 相机坐标系（针孔模型）
    z_c = depth_mm / 1000.0
    x_c = (px - self.cx) * z_c / self.fx
    y_c = (py - self.cy) * z_c / self.fy

    # 2. 相机坐标系 → 机器人坐标系
    # 相机向前下方倾斜 CAMERA_TILT_ANGLE (45°)
    theta = np.radians(self.CAMERA_TILT_ANGLE)
    ground_x = (z_c * np.cos(theta)) - (y_c * np.sin(theta)) + self.CAMERA_OFFSET_X
    ground_y = -x_c
    vertical_drop = (y_c * np.cos(theta)) + (z_c * np.sin(theta))
    ground_z = max(0.01, min(self.CAMERA_HEIGHT - vertical_drop, 0.25))
    return [ground_x, ground_y, ground_z]
```



Qwen的应用：工具一、视觉



同濟大學
TONGJI UNIVERSITY

```
def _run_tool_calling(self, query):
    log = self.get_logger().info

    # 【阶段 1】大模型输入
    log(f' query : "{query}"')
    log(f' tools : [detect_blocks]')

    # 【阶段 2】大模型输出
    result = self.llm_client.llm_tools(
        query, DETECT_ONLY_PROMPT, self.llm_model,
        [DETECT_TOOL], None)
    content, tool_call = result[0], result[1]
    if not tool_call:
        return
    tool_id, tool_name, tool_args = tool_call[0], tool_call[1], tool_call[2]
    log(f' tool_call: name={tool_name}, args={tool_args}')
```

```
    # 【阶段 3】执行工具
    detection_json = self.execute_detect_blocks()
    log(f' 工具输出 : {detection_json}')
```

```
    # 【阶段 4】结果返回大模型
    final = self.llm_client.llm_tools_result(tool_id, detection_json)
```

```
    # 【阶段 5】最终回答
    log(f' 大模型回答: {final[0]}')
```



测试过程

1. 将相关文件通过winscp拖入home/pi/docker/tmp/
2. 打开VNC Viewer，连接raspberrypi.local，新建一个终端，逐行输入：

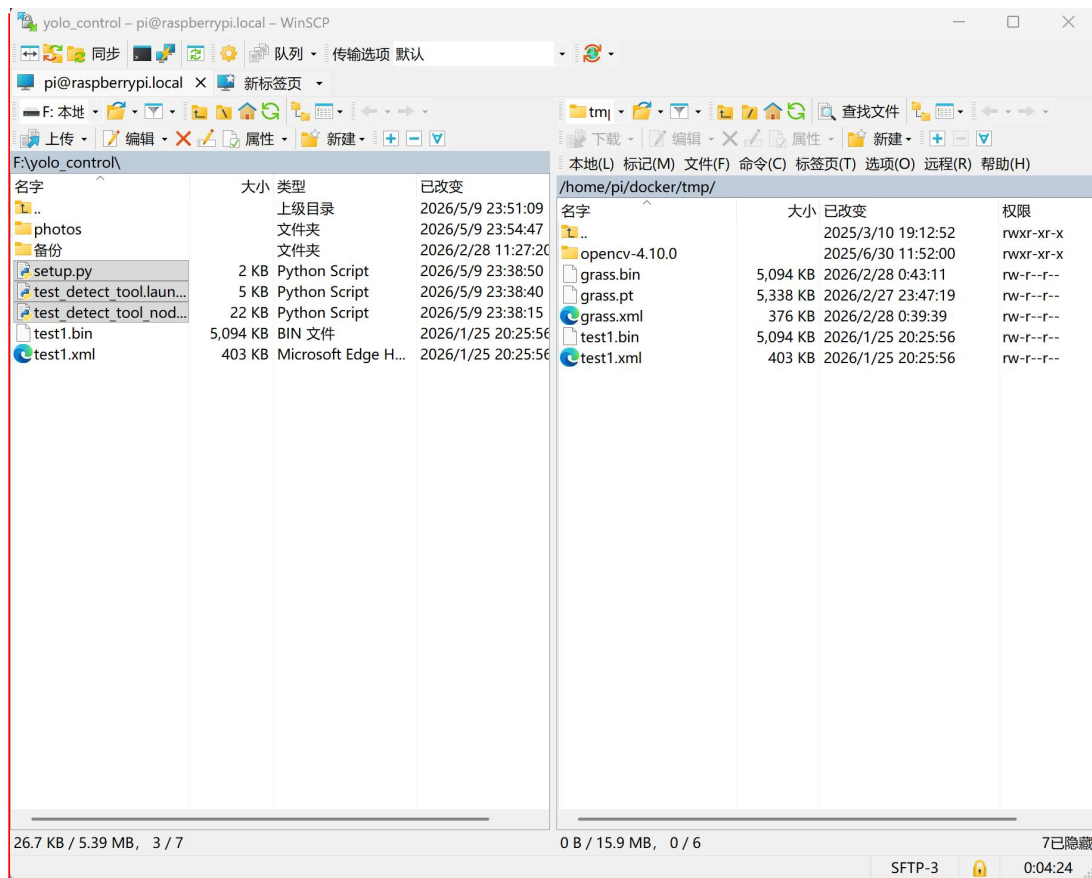
```
cp ~/shared/test_detect_tool_node.py ~/ros2_ws/src/yolov11_detect/yolov11_detect/  
cp ~/shared/test_detect_tool.launch.py ~/ros2_ws/src/yolov11_detect/launch/  
cp ~/shared/setup.py ~/ros2_ws/src/yolov11_detect/  
chmod +x ~/ros2_ws/src/yolov11_detect/yolov11_detect/test_detect_tool_node.py  
cd ~/ros2_ws  
colcon build --packages-select yolov11_detect
```
3. 第一个终端中输入ros2 launch yolov11_detect test_detect_tool.launch.py，等待指令完成
4. 放置方块，然后在第二个终端中输入（不换行）

```
ros2 topic pub --once /test_detect_tool/user_query std_msgs/msg/String  
"data: 'What blocks are in front of you?'"
```



测试过程

1. 将相关文件通过winscp拖入home/pi/docker/tmp/





测试过程

2. 打开VNC Viewer，连接raspberrypi.local，新建一个终端，逐行输入：

```
cp ~/shared/test_detect_tool_node.py ~/ros2_ws/src/yolov11_detect/yolov11_detect/
```

```
cp ~/shared/test_detect_tool.launch.py ~/ros2_ws/src/yolov11_detect/launch/
```

```
cp ~/shared/setup.py ~/ros2_ws/src/yolov11_detect/
```

```
chmod +x ~/ros2_ws/src/yolov11_detect/yolov11_detect/test_detect_tool_node.py
```

```
cd ~/ros2_ws
```

```
colcon build --packages-select yolov11_detect
```

```
/bin/zsh
/bin/zsh 80x24
当前环境是ROS2|The current environment is ROS2
-----
LIDAR:    MS200
CAMERA:   aurora
MIC_TYPE: WonderEchoPro
MACHINE:  LanderP1_Mecanum
HOST:     /
MASTER:   /
ROS_DOMAIN_ID: 0
VERSION:  |V1.0.0|2025-11-14|
zsh: corrupt history file /home/ubuntu/.zsh/.zsh_history
❏ | * ~ ▶ 16:19:52 〇
```




测试过程

3. 第一个终端中输入`ros2 launch yolov11_detect test_detect_tool.launch.py`，等待指令完成

```
/bin/zsh
/bin/zsh 80x24
-----
当前环境是ROS2|The current environment is ROS2
-----
LIDAR: MS200
CAMERA: aurora
MIC_TYPE: WonderEchoPro
MACHINE: LanderPi_Mecanum
HOST: /
MASTER: /
ROS_DOMAIN_ID: 0
VERSION: |V1.0.0|2025-11-14|
zsh: corrupt history file /home/ubuntu/.zsh/.zsh_history
> ~/.stop_ros.sh
zsh: killed ~/.stop_ros.sh
KILL x | 16:36:45
ros2 launch yolov11_detect test_detect_tool.launch.py
```




测试过程

4. 放置方块，然后在第二个终端中输入

```
ros2 topic pub --once /test_detect_tool/user_query std_msgs/msg/String
```

```
"data: 'What blocks are in front of you?'"
```

```
raspberrypi.local (WayVNC) - VNC Viewer
/bin/zsh
/bin/zsh
/bin/zsh 80x17
[test_detect_tool-11] [INFO] [1778344677.862354155] [test_detect_tool]:
[test_detect_tool-11] [INFO] [1778344677.862967078] [test_detect_tool]: 使用方
法（在另一个终端执行）：
[test_detect_tool-11] [INFO] [1778344677.863702205] [test_detect_tool]:
[test_detect_tool-11] [INFO] [1778344677.864255480] [test_detect_tool]: 方法1
- 大模型 Tool Calling 测试：
[test_detect_tool-11] [INFO] [1778344677.864793274] [test_detect_tool]: ros2
topic pub --once /test_detect_tool/user_query std_msgs/msg/String "data: '看看
桌上有什么方块'"
[test_detect_tool-11] [INFO] [1778344677.866931302] [test_detect_tool]:
[test_detect_tool-11] [INFO] [1778344677.868455407] [test_detect_tool]: 方法2
- 仅视觉检测（不经过大模型）：
[test_detect_tool-11] [INFO] [1778344677.869259792] [test_detect_tool]: ros2
service call /test_detect_tool/detect_only std_srvs/srv/Trigger
[test_detect_tool-11] [INFO] [1778344677.870061548] [test_detect_tool]: =====
=====
/bin/zsh
/bin/zsh 160x24
当前环境是ROS2|The current environment is ROS2
-----
LIDAR: MS200
CAMERA: aurora
MIC TYPE: WonderEchoPro
MACHINE: LanderPi_Mecanum
HOST: /
MASTER: /
ROS_DOMAIN_ID: 0
VERSION: |V1.0.0|2025-11-14|
ros2 topic pub --once /test_detect_tool/user_query std_msgs/msg/String "data: 'what are these'"
16:37:49
```



Qwen的应用：工具一、视觉



同济大学
TONGJI UNIVERSITY

测试结果

```
raspberrypi.local (WayVNC) - VNC Viewer
/bin/zsh
/bin/zsh 157x17
[test_detect_tool-11] [INFO] [1778344877.598971481] [test_detect_tool]:
[test_detect_tool-11] -----
[test_detect_tool-11] [INFO] [1778344877.599464607] [test_detect_tool]: 【阶段 5】大模型最终回答
[test_detect_tool-11] [INFO] [1778344877.599948529] [test_detect_tool]: -----
[test_detect_tool-11] [INFO] [1778344877.600439062] [test_detect_tool]: I detected two yellow blocks on the workbench:
[test_detect_tool-11]
[test_detect_tool-11] 1. **Yellow block 1**
[test_detect_tool-11]    - Confidence: 93.2%
[test_detect_tool-11]    - Position: X=0.203m, Y=0.006m, Z=0.01m
[test_detect_tool-11]
[test_detect_tool-11] 2. **Yellow block 2**
[test_detect_tool-11]    - Confidence: 92.2%
[test_detect_tool-11]    - Position: X=0.17m, Y=-0.05m, Z=0.01m
[test_detect_tool-11]
[test_detect_tool-11] Summary: There are two yellow blocks (2cm x 2cm x 2cm) on the workbench, both located in front of the robot and slightly to the right.
They are approximately at ground level (Z=0.01m).
█

/bin/zsh
/bin/zsh 156x24
-----
当前环境是ROS2|The current environment is ROS2
-----
LIDAR:    MS200
CAMERA:   aurora
MIC_TYPE: WonderEchoPro
MACHINE:  LanderPi_Mecanum
HOST:     /
MASTER:   /
ROS_DOMAIN_ID: 0
VERSION:  |V1.0.0|2025-11-14|
zsh: corrupt history file /home/ubuntu/.zsh/.zsh_history
> ros2 topic pub --once /test_detect_tool/user_query std_msgs/msg/String "data: 'what are these'"
waiting for at least 1 matching subscription(s)...
publisher: beginning loop
publishing #1: std_msgs.msg.String(data='what are these')
```

工具二：抓取

【工具要求】

- 编写一个 ROS 2 Python 脚本 `test_grasp_tool_node.py`，初始化并创建一个名为 `test_grasp_tool` 的节点。
- 定义抓取工具 `GRASP_TOOL`：工具名为 `grasp_object`，必填参数 `x, y, z`（单位：米），可选参数 `pitch`（默认 -90° ，垂直向下抓取）。
- 编写系统提示词 `GRASP_ONLY_PROMPT`，告诉大模型坐标范围、参数含义、典型示例。
- 实现工具函数 `execute_grasp_object(x, y, z, pitch=-90)`：依次执行 张爪 → 移到目标上方 4cm → 下降 → 闭爪 → 抬起 → 复位；通过 IK 服务 `/kinematics/set_pose_target` 求解关节角，通过话题 `/ros_robot_controller/bus_servo/set_position` 控制夹爪舵机（ID=10，张开=200，闭合=495）。
- 创建一个订阅者订阅话题 `~/user_query`（`std_msgs/msg/String`，队列长度 1），在回调中起新线程执行 5 阶段 Tool Calling 流程。
- 工具函数最终返回 JSON 字符串 `{success, position, pitch, message}`，再交给大模型生成最终回答。



Qwen的应用：工具二、抓取



同濟大學
TONGJI UNIVERSITY

```
GRASP_TOOL = {
  "type": "function",
  "function": {
    "name": "grasp_object",
    "description": (
      "控制机械臂抓取指定坐标处的物体。"
      "机械臂会移动到目标上方，下降，夹取，然后抬起。"
      "坐标系：X 轴向前(0.05-0.30m)，Y 轴向左(-0.15-0.15m)，"
      "Z 轴向上(0.01-0.25m)。"
    ),
    "parameters": {
      "type": "object",
      "properties": {
        "x": {"type": "number",
              "description": "X坐标 (米)，范围0.05~0.30"},
        "y": {"type": "number",
              "description": "Y坐标 (米)，范围-0.15~0.15"},
        "z": {"type": "number",
              "description": "Z坐标 (米)，范围0.01~0.25"},
        "pitch": {"type": "number",
                  "description": "抓取角度 (度)，-90为垂直向下，默认-90"}
      },
      "required": ["x", "y", "z"]
    }
  }
}
```



Qwen的应用：工具二、抓取



同濟大學
TONGJI UNIVERSITY

```
GRASP_ONLY_PROMPT = """你是一个机器人抓取助手。你有一个工具 grasp_object，  
可以控制机械臂抓取指定坐标处的物体。
```

```
## 坐标系（机器人底盘地面投影为原点）
```

- X 轴：机器人前方为正（范围 0.05~0.30 米）
- Y 轴：机器人左侧为正（范围 -0.15~0.15 米）
- Z 轴：向上为正（地面方块 $z \approx 0.01$ 米）

```
## 参数说明
```

- x, y, z: 目标方块的 3D 坐标（米），必填
- pitch: 抓取角度（度），-90 为垂直向下，可选，默认 -90

```
## 使用规则
```

1. 用户给出坐标后直接调用工具
2. 如果坐标超出范围，提醒用户并给出合理范围
3. 调用工具后根据返回结果告知用户抓取是否成功

```
## 示例
```

```
用户说"抓取坐标(0.15, 0.05, 0.01)的方块"
```

```
→ 调用 grasp_object(x=0.15, y=0.05, z=0.01)
```

```
用户说"抓取正前方0.2米处地面的方块"
```

```
→ 调用 grasp_object(x=0.20, y=0.0, z=0.01)
```

```
"""
```




Qwen的应用：工具二、抓取



同濟大學
TONGJI UNIVERSITY

```
def execute_grasp_object(self, x, y, z, pitch=-90.0):
    # 0. 坐标安全裁剪
    x = max(0.05, min(x, 0.30))
    y = max(-0.15, min(y, 0.15))
    z = max(0.01, min(z, 0.25))

    # 1. 张开夹爪
    self.set_gripper('open')

    # 2. 移到目标上方 4cm (IK 求解 + 发布舵机指令)
    if not self.move_to_target(x, y, z + 0.04, pitch, duration=1.5):
        self.move_to_home_raw()
        return json.dumps({"success": False, "error": "无法到达准备点"},
                           ensure_ascii=False)

    # 3. 下降到抓取点
    self.move_to_target(x, y, z, pitch, duration=0.8)

    # 4. 关闭夹爪夹紧物体
    self.set_gripper('close')

    # 5. 抬起物体
    self.move_to_target(x, y, z + 0.06, pitch, duration=1.0)

    # 6. 回到初始位置 (保持夹爪闭合)
    self._move_arm_home_keep_gripper()

    return json.dumps({
        "success": True,
        "position": [round(x, 3), round(y, 3), round(z, 3)],
        "pitch": pitch,
        "message": f"已抓取({x:.3f},{y:.3f},{z:.3f})处的物体"
    }, ensure_ascii=False)
```



Qwen的应用：工具二、抓取



同濟大學
TONGJI UNIVERSITY

```
def set_gripper(self, state):  
    """夹爪舵机: ID=10, 张开=200, 闭合=495 (已调参, 安全值) """  
    msg = RosServosPosition()  
    msg.duration = 0.5  
    pos_value = 200 if state == 'open' else 495  
    s = ServoPosition(id=10, position=pos_value)  
    msg.position.append(s)  
    self.arm_pub.publish(msg)  
    time.sleep(0.8)
```




Qwen的应用：工具二、抓取



同濟大學
TONGJI UNIVERSITY

```
req = SetRobotPose.Request()
req.position    = [float(x), float(y), float(z)]
req.pitch      = float(pitch)
req.pitch_range = [-180.0, 180.0]
req.resolution  = 1.0
future = self.set_pose_target_client.call_async(req)
# ...等待 future 完成, response.pulse 即为 5 个关节舵机的脉冲值
```



```
result = self.llm_client.llm_tools(  
    query, GRASP_ONLY_PROMPT, self.llm_model, [GRASP_TOOL], None)  
content, tool_call = result[0], result[1]  
if tool_call:  
    tool_id, tool_name, tool_args = tool_call[0], tool_call[1], tool_call[2]  
    x = float(tool_args.get('x', 0.15))  
    y = float(tool_args.get('y', 0.0))  
    z = float(tool_args.get('z', 0.01))  
    pitch = float(tool_args.get('pitch', -90.0))  
    grasp_result = self.execute_grasp_object(x, y, z, pitch)  
    final = self.llm_client.llm_tools_result(tool_id, grasp_result)
```



测试过程

1. 将相关文件通过winscp拖入home/pi/docker/tmp/
2. 打开VNC Viewer，连接raspberrypi.local，新建一个终端，逐行输入：

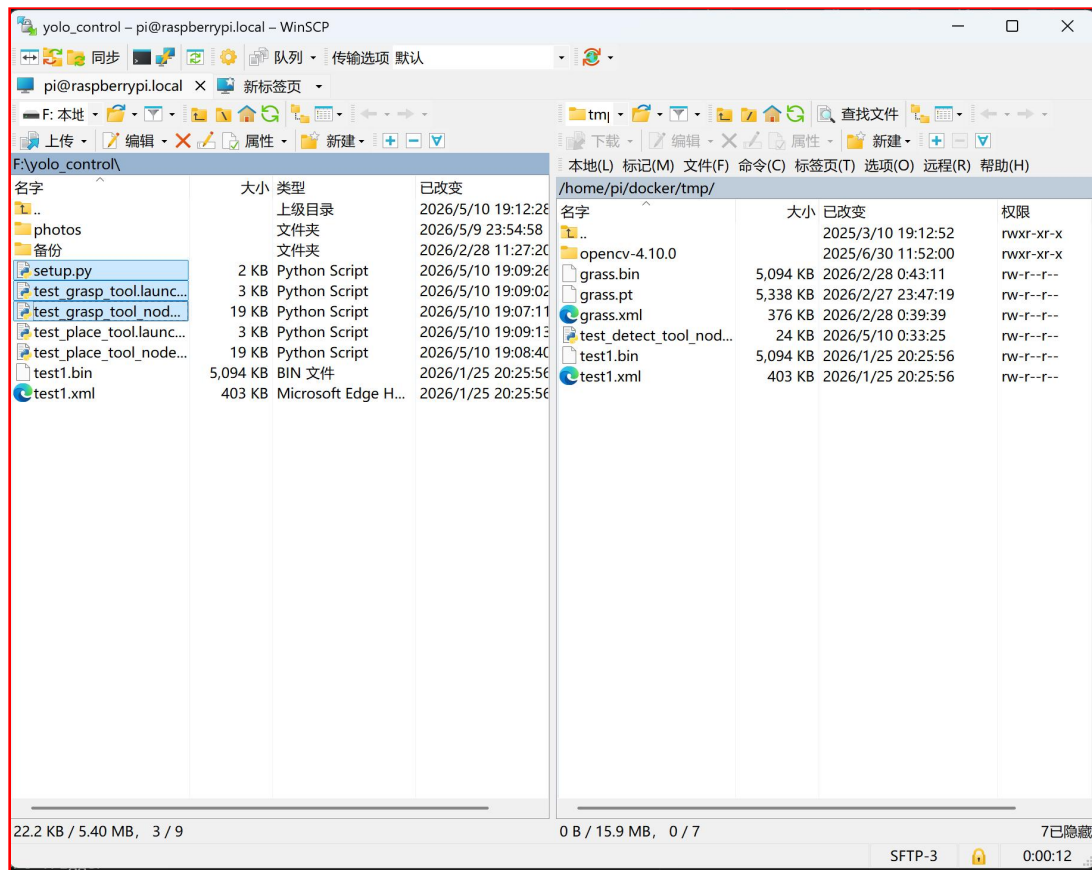
```
cp ~/shared/test_grasp_tool_node.py ~/ros2_ws/src/yolov11_detect/yolov11_detect/  
cp ~/shared/test_grasp_tool.launch.py ~/ros2_ws/src/yolov11_detect/launch/  
cp ~/shared/setup.py ~/ros2_ws/src/yolov11_detect/  
chmod +x ~/ros2_ws/src/yolov11_detect/yolov11_detect/test_grasp_tool_node.py  
cd ~/ros2_ws  
colcon build --packages-select yolov11_detect
```
3. 第一个终端中输入ros2 launch yolov11_detect test_grasp_tool.launch.py，等待指令完成
4. 在第二个终端中输入（不换行）

```
ros2 topic pub --once /test_grasp_tool/user_query std_msgs/msg/String  
"data: 'grasp the block(0.15, 0.05, 0.01)'"
```



测试过程

1. 将相关文件通过winscp拖入home/pi/docker/tmp/





测试过程

2. 打开VNC Viewer，连接raspberrypi.local，新建一个终端，逐行输入：

```
cp ~/shared/test_grasp_tool_node.py ~/ros2_ws/src/yolov11_detect/yolov11_detect/
```

```
cp ~/shared/test_grasp_tool.launch.py ~/ros2_ws/src/yolov11_detect/launch/
```

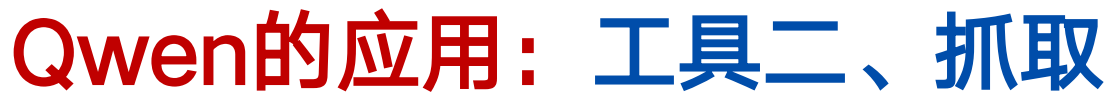
```
cp ~/shared/setup.py ~/ros2_ws/src/yolov11_detect/
```

```
chmod +x ~/ros2_ws/src/yolov11_detect/yolov11_detect/test_grasp_tool_node.py
```

```
cd ~/ros2_ws
```

```
colcon build --packages-select yolov11_detect
```

```
/bin/zsh
/bin/zsh 80x24
当前环境是ROS2|The current environment is ROS2
-----
LIDAR:    MS200
CAMERA:   aurora
MIC_TYPE: WonderEchoPro
MACHINE:  LanderP1_Mecanum
HOST:     /
MASTER:   /
ROS_DOMAIN_ID: 0
VERSION:  |V1.0.0|2025-11-14|
zsh: corrupt history file /home/ubuntu/.zsh/.zsh_history
❏ | 16:19:52 〇
```



3. 第一个终端中输入 `ros2 launch yolov11_detect test_grasp_tool.launch.py`, 等待指令完成

```
当前环境是ROS2|The current environment is ROS2
-----
LIDAR:    MS200
CAMERA:   aurora
VIC_TYPE: WonderEchoPro
MACHINE:  LanderPi_Mecanum
HOST:     /
MASTER:   /
ROS_DOMAIN_ID: 0
VERSION:  |v1.0.0|2025-11-14|
zsh: corrupt history file /home/ubuntu/.zsh/.zsh_history
> ~/.stop_ros.sh
zsh: killed      ~/.stop_ros.sh
[ros2 launch yotlov11_detect test_grasp_tool.launch.py
```



Qwen的应用：工具二、抓取



同济大学
TONGJI UNIVERSITY

测试过程

4. 在第二个终端中输入（不换行）

```
ros2 topic pub --once /test_grasp_tool/user_query std_msgs/msg/String
```

```
"data: 'grasp the block(0.15, 0.05, 0.01)'"
```

```
raspberrypi.local (WayVNC) - VNC Viewer
/bin/zsh
/bin/zsh
[1778411839.675705293] [test_grasp_tool]: LLM 客户端初始化成功, 模型: qwen-max-latest
[1778411839.700086171] [test_grasp_tool]: 机械臂控制初始化完成
[1778411839.708947448] [test_grasp_tool]:
=====
[1778411839.710138646] [test_grasp_tool]: 抓取工具单独测试 (大模型 Tool Calling)
[1778411839.710602626] [test_grasp_tool]:
[1778411839.719980143] [test_grasp_tool]: 大模型已就绪
[1778411839.711339438] [test_grasp_tool]:
使用方法 (在另一个终端执行):
[1778411839.711684732] [test_grasp_tool]:
[1778411839.712244656] [test_grasp_tool]: 方法1 - 大模型 Tool Calling 测试:
[1778411839.713074208] [test_grasp_tool]: ros2 topic pub --once /test_grasp_tool/user_query std_msgs/msg/String "data: '抓取坐标(0.15,
0.05, 0.01)处的方块'"
[1778411839.713963889] [test_grasp_tool]:
[1778411839.714748090] [test_grasp_tool]: 方法2 - 直接抓取测试 (固定坐标, 不经过大模型):
[1778411839.715856844] [test_grasp_tool]: ros2 service call /test_grasp_tool/grasp_test std_srvs/srv/Trigger
[1778411839.716922525] [test_grasp_tool]:
[1778411839.717770995] [test_grasp_tool]:
[1778411839.718534018] [test_grasp_tool]: [TestGraspTool] 节点启动完成

当前环境是ROS2|The current environment is ROS2
LIDAR: MS200
CAMERA: aurora
MIC_TYPE: WonderEchoPro
MACHINE: LanderPi_Mecanum
HOST: /
MASTER: /
ROS_DOMAIN_ID: 0
VERSION: |V1.0.0|2025-11-14|
zsh: corrupt history file /home/ubuntu/.zsh/.zsh_history
[ros2] topic pub --once /test_grasp_tool/user_query std_msgs/msg/String "data: 'grasp the block(0.15, 0.05, 0.01)'"
```




Qwen的应用：工具二、抓取



同济大学
TONGJI UNIVERSITY

测试结果

```
raspberrypi.local (WayVNC) - VNC Viewer
/bin/zsh
/bin/zsh
/bin/zsh 166x44
[test_grasp_tool-9] [INFO] [1778412087.374446991] [test_grasp_tool]: 收到用户指令: grasp the block(0.15, 0.05, 0.01)
[test_grasp_tool-9] [INFO] [1778412087.375543061] [test_grasp_tool]:
[test_grasp_tool-9] [INFO] [1778412087.375997678] [test_grasp_tool]: 【阶段 1】大模型输入
[test_grasp_tool-9] [INFO] [1778412087.376424641] [test_grasp_tool]:
[test_grasp_tool-9] [INFO] [1778412087.376822597] [test_grasp_tool]: query : "grasp the block(0.15, 0.05, 0.01)"
[test_grasp_tool-9] [INFO] [1778412087.377239150] [test_grasp_tool]: prompt : (系统提示词, 493 字)
[test_grasp_tool-9] [INFO] [1778412087.377699861] [test_grasp_tool]: model : qwen-max-latest
[test_grasp_tool-9] [INFO] [1778412087.378172277] [test_grasp_tool]: tools : [grasp_object] (参数: x, y, z, pitch)
[test_grasp_tool-9] [INFO] [1778412089.740096985] [test_grasp_tool]:
[test_grasp_tool-9] [INFO] [1778412089.740709130] [test_grasp_tool]: 【阶段 2】大模型输出
[test_grasp_tool-9] [INFO] [1778412089.741265653] [test_grasp_tool]:
[test_grasp_tool-9] [INFO] [1778412089.741827343] [test_grasp_tool]: content : (无, 大模型选择了调用工具)
[test_grasp_tool-9] [INFO] [1778412089.742519912] [test_grasp_tool]: tool_call :
[test_grasp_tool-9] [INFO] [1778412089.743171825] [test_grasp_tool]: id : call_26506eb910214313b260c4
[test_grasp_tool-9] [INFO] [1778412089.743851410] [test_grasp_tool]: name : grasp_object
[test_grasp_tool-9] [INFO] [1778412089.744742298] [test_grasp_tool]: arguments : {"x": 0.15, "y": 0.05, "z": 0.01}
[test_grasp_tool-9] [INFO] [1778412089.745536925] [test_grasp_tool]:
[test_grasp_tool-9] [INFO] [1778412089.746195617] [test_grasp_tool]: 【阶段 3】执行抓取工具 grasp_object
[test_grasp_tool-9] [INFO] [1778412089.746776793] [test_grasp_tool]:
[test_grasp_tool-9] [INFO] [1778412089.747353060] [test_grasp_tool]: 工具输入 : x=0.15, y=0.05, z=0.01, pitch=-90.0
[test_grasp_tool-9] [INFO] [1778412089.747917362] [test_grasp_tool]: 执行流程 : 张爪 → 移到上方 → 下降 → 夹取 → 抬起 → 复位
[test_grasp_tool-9] [INFO] [1778412089.748501557] [test_grasp_tool]: 执行中...
[test_grasp_tool-9] [INFO] [1778412089.749156508] [test_grasp_tool]: 开始抓取: (0.150, 0.050, 0.010), pitch=-90.0
[test_grasp_tool-9] [INFO] [1778412090.550854328] [test_grasp_tool]: 夹爪: open
[test_grasp_tool-9] [INFO] [1778412094.276416014] [test_grasp_tool]: 夹爪: close
[test_grasp_tool-9] [INFO] [1778412097.791246205] [test_grasp_tool]: 抓取完成: (0.150, 0.050, 0.010)
[test_grasp_tool-9] [INFO] [1778412097.791759498] [test_grasp_tool]: 耗时 : 8.0 秒
[test_grasp_tool-9] [INFO] [1778412097.792176533] [test_grasp_tool]: 工具输出 : {"success": true, "position": [0.15, 0.05, 0.01], "pitch": -90.0, "message": "已抓取"}
(0.150, 0.050, 0.010)处的物体, 当前夹爪闭合持物"}
[test_grasp_tool-9] [INFO] [1778412097.792623481] [test_grasp_tool]:
[test_grasp_tool-9] [INFO] [1778412097.793060705] [test_grasp_tool]: 【阶段 4】将工具结果返回大模型
[test_grasp_tool-9] [INFO] [1778412097.793446587] [test_grasp_tool]:
[test_grasp_tool-9] [INFO] [1778412097.793814243] [test_grasp_tool]: tool_call_id : call_26506eb910214313b260c4
[test_grasp_tool-9] [INFO] [1778412097.794194661] [test_grasp_tool]: 调用 llm_tools_result ...
[test_grasp_tool-9] [INFO] [1778412104.406275811] [test_grasp_tool]:
[test_grasp_tool-9] [INFO] [1778412104.406798091] [test_grasp_tool]: 【阶段 5】大模型最终回答
[test_grasp_tool-9] [INFO] [1778412104.407326632] [test_grasp_tool]:
[test_grasp_tool-9] [INFO] [1778412104.409350634] [test_grasp_tool]: The block at coordinates (0.15, 0.05, 0.01) has been successfully grasped.
```

工具三：放置

【工具要求】

- 编写一个 ROS 2 Python 脚本 `test_place_tool_node.py`，初始化并创建一个名为 `test_place_tool` 的节点。
- 定义放置工具 `PLACE_TOOL`：工具名为 `place_object`，必填参数 `x, y, z`（单位：米）。
- 编写系统提示词 `PLACE_ONLY_PROMPT`，内置方位对照表（正前方/左前方/右前方/远处/近处 → 坐标）、堆叠规则（第 1 层 $z=0.01$ 、第 2 层 $z=0.03$ 、第 3 层 $z=0.05$ ）。
- 实现工具函数 `execute_place_object(x, y, z)`：依次执行 移到上方 6cm → 下降到 $z+0.01$ → 松爪释放 → 抬升到 0.12m → 复位。
- 创建订阅者订阅 `~/user_query`（`std_msgs/msg/String`，队列长度 1），在回调中起新线程执行 5 阶段 Tool Calling 流程。



Qwen的应用：工具三、放置



同濟大學
TONGJI UNIVERSITY

```
PLACE_TOOL = {
  "type": "function",
  "function": {
    "name": "place_object",
    "description": (
      "将手中的物体放置到指定坐标。"
      "机械臂会移动到目标上方，下降到放置高度，松开夹爪释放物体。"
      "堆叠规则：第1层z=0.01，第2层z=0.03，第3层z=0.05（每层+0.02m）。"
      "坐标系：X 轴向前(0.05-0.30m)，Y 轴向左(-0.15-0.15m)，"
      "Z 轴向上(0.01-0.25m)。"
    ),
    "parameters": {
      "type": "object",
      "properties": {
        "x": {"type": "number", "description": "放置X坐标（米），0.05~0.30"},
        "y": {"type": "number", "description": "放置Y坐标（米），-0.15~0.15"},
        "z": {"type": "number",
              "description": "放置Z坐标（米），第1层=0.01，第2层=0.03，第3层=0.05"}
      },
      "required": ["x", "y", "z"]
    }
  }
}
```



Qwen的应用：工具三、放置



同濟大學
TONGJI UNIVERSITY

```
PLACE_ONLY_PROMPT = """你是一个机器人放置助手。你有一个工具 place_object，  
可以将机械臂夹爪中的物体放置到指定坐标。
```

```
## 坐标系（机器人底盘地面投影为原点）
```

- X 轴：机器人前方为正（范围 0.05~0.30 米）
- Y 轴：机器人左侧为正（范围 -0.15~0.15 米）
- Z 轴：向上为正（范围 0.01~0.25 米）

```
## 堆叠规则
```

- 方块高度 2cm (0.02m)
- 第 1 层（地面）：z = 0.01
- 第 2 层：z = 0.03
- 第 3 层：z = 0.05

```
## 方位对照（自然语言 → 坐标）
```

- "正前方" → x ≈ 0.15, y = 0.00
- "左前方" → x ≈ 0.15, y ≈ 0.10
- "右前方" → x ≈ 0.15, y ≈ -0.10
- "远处/前面远一点" → x ≈ 0.25
- "近处/前面近一点" → x ≈ 0.08

```
## 示例
```

```
用户说"放到正前方地面" → place_object(x=0.15, y=0.0, z=0.01)  
用户说"放到左边" → place_object(x=0.15, y=0.10, z=0.01)  
用户说"堆到第二层" → place_object(x=0.15, y=0.0, z=0.03)
```

```
"""
```




Qwen的应用：工具三、放置



同濟大學
TONGJI UNIVERSITY

```
def execute_place_object(self, x, y, z, pitch=-90.0):
    x = max(0.05, min(x, 0.30))
    y = max(-0.15, min(y, 0.15))
    z = max(0.01, min(z, 0.25))

    # 1. 移到放置位置上方 6cm
    if not self.move_to_target(x, y, z + 0.06, pitch, duration=1.8):
        self.move_to_home_raw()
        return json.dumps({"success": False, "error": "无法到达放置区上方"},
                           ensure_ascii=False)

    # 2. 下降到放置高度 (+1cm 偏移, 方块自由落下约 1cm)
    self.move_to_target(x, y, z + 0.01, pitch, duration=1.0)

    # 3. 打开夹爪释放物体
    self.set_gripper('open')

    # 4. 垂直抬升到安全高度
    self.move_to_target(x, y, 0.12, pitch, duration=1.0)

    # 5. 回到初始位置
    self.move_to_home_raw(duration=2.0)

    return json.dumps({
        "success": True,
        "position": [round(x, 3), round(y, 3), round(z, 3)],
        "message": f"已放置到({x:.3f},{y:.3f},{z:.3f})"
    }, ensure_ascii=False)
```



Qwen的应用：工具三、放置



同濟大學
TONGJI UNIVERSITY

```
def _run_tool_calling(self, query):
    # 【阶段 2】调用大模型，传入 [PLACE_TOOL]
    result = self.llm_client.llm_tools(
        query, PLACE_ONLY_PROMPT, self.llm_model, [PLACE_TOOL], None)
    content, tool_call = result[0], result[1]
    if not tool_call:
        return

    tool_id, tool_name, tool_args = tool_call[0], tool_call[1], tool_call[2]

    # 【阶段 3】执行工具
    x = float(tool_args.get('x', 0.15))
    y = float(tool_args.get('y', 0.0))
    z = float(tool_args.get('z', 0.01))
    place_result = self.execute_place_object(x, y, z)

    # 【阶段 4】结果返回大模型
    final = self.llm_client.llm_tools_result(tool_id, place_result)

    # 【阶段 5】打印大模型最终回答
    self.get_logger().info(f'大模型回答: {final[0]}')
```



测试过程

1. 将相关文件通过winscp拖入home/pi/docker/tmp/
2. 打开VNC Viewer，连接raspberrypi.local，新建一个终端，逐行输入：

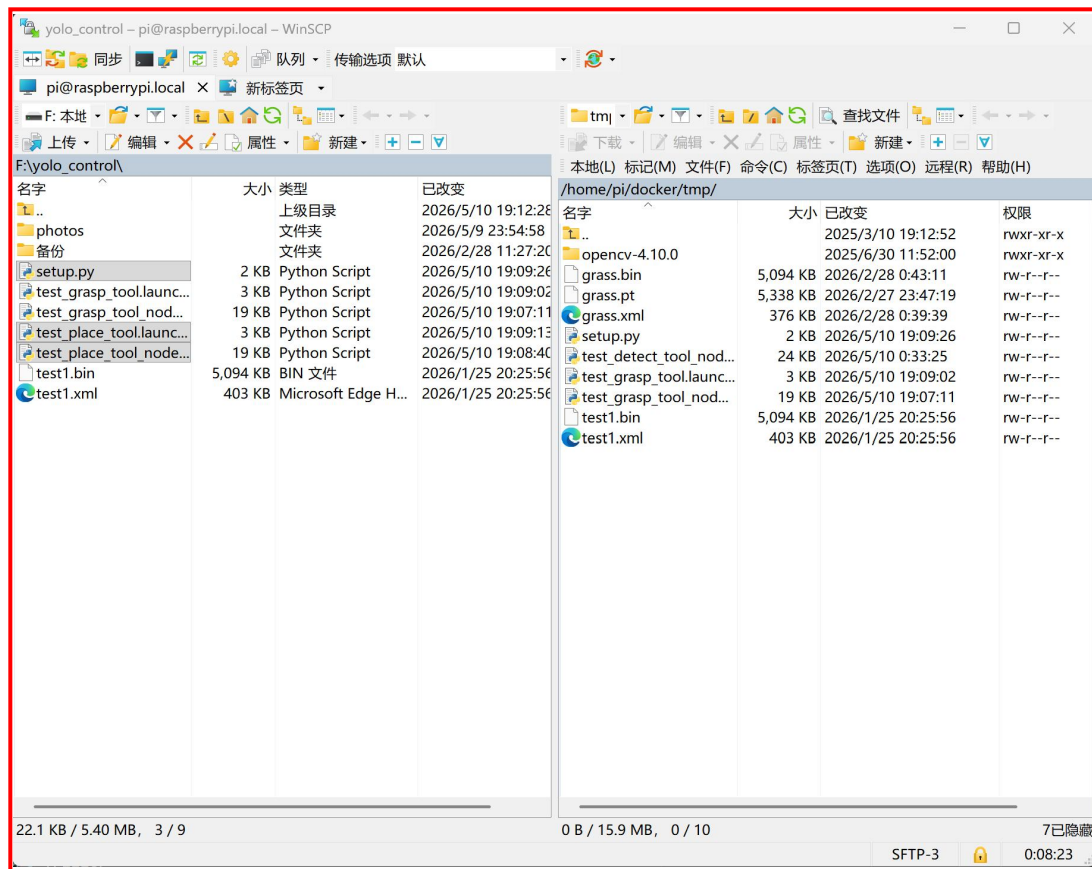
```
cp ~/shared/test_place_tool_node.py ~/ros2_ws/src/yolov11_detect/yolov11_detect/  
cp ~/shared/test_place_tool.launch.py ~/ros2_ws/src/yolov11_detect/launch/  
cp ~/shared/setup.py ~/ros2_ws/src/yolov11_detect/  
chmod +x ~/ros2_ws/src/yolov11_detect/yolov11_detect/test_place_tool_node.py  
cd ~/ros2_ws  
colcon build --packages-select yolov11_detect
```
3. 第一个终端中输入ros2 launch yolov11_detect test_place_tool.launch.py，等待指令完成
4. 在第二个终端中输入（不换行）

```
ros2 topic pub --once /test_place_tool/user_query std_msgs/msg/String  
"data: 'place the block to the front right'"
```




测试过程

1. 将相关文件通过winscp拖入home/pi/docker/tmp/





测试过程

2. 打开VNC Viewer，连接raspberrypi.local，新建一个终端，逐行输入：

```
cp ~/shared/test_place_tool_node.py ~/ros2_ws/src/yolov11_detect/yolov11_detect/
```

```
cp ~/shared/test_place_tool.launch.py ~/ros2_ws/src/yolov11_detect/launch/
```

```
cp ~/shared/setup.py ~/ros2_ws/src/yolov11_detect/
```

```
chmod +x ~/ros2_ws/src/yolov11_detect/yolov11_detect/test_place_tool_node.py
```

```
cd ~/ros2_ws
```

```
colcon build --packages-select yolov11_detect
```

```
/bin/zsh
/bin/zsh 80x24
当前环境是ROS2|The current environment is ROS2
-----
LIDAR:    MS200
CAMERA:   aurora
MIC_TYPE: WonderEchoPro
MACHINE:  LanderP1_Mecanum
HOST:     /
MASTER:   /
ROS_DOMAIN_ID: 0
VERSION:  |V1.0.0|2025-11-14|
zsh: corrupt history file /home/ubuntu/.zsh/.zsh_history
❏ | * ~ ▶
```



测试过程

3. 第一个终端中输入`ros2 launch yolov11_detect test_place_tool.launch.py`，等待指令完成

```
-----  
当前环境是ROS2|The current environment is ROS2  
-----  
LIDAR:    MS200  
CAMERA:   aurora  
MIC_TYPE: WonderEchoPro  
MACHINE:  LanderPi_Mecanum  
HOST:     /  
MASTER:   /  
ROS_DOMAIN_ID: 0  
VERSION:  |V1.0.0|2025-11-14|  
zsh: corrupt history file /home/ubuntu/.zsh/.zsh_history  
[ ~ ]  
[ ros2 launch yolov11_detect test_place_tool.launch.py ]
```



Qwen的应用：工具三、放置



同济大学
TONGJI UNIVERSITY

测试过程

4. 在第二个终端中输入（不换行）

```
ros2 topic pub --once /test_place_tool/user_query std_msgs/msg/String
```

```
"data: 'place the block to the front right'"
```

The screenshot shows two terminal windows. The top window, titled 'raspberrypi.local (WayVNC) - VNC Viewer', displays the output of the 'test_place_tool' node. It shows several INFO messages with timestamps and IDs, followed by a note in Chinese: '注意：请确保夹具已夹住物体（先运行抓取测试）' (Note: Please ensure the gripper has clamped the object (run the grasping test first)). Below this, it provides usage instructions: '使用方法（在另一个终端执行）：' (Usage method (execute in another terminal):). Two methods are listed: '方法1 - 大模型 Tool Calling 测试：' (Method 1 - Large model Tool Calling test) and '方法2 - 直接放置测试（固定坐标，不经过大模型）：' (Method 2 - Direct placement test (fixed coordinates, no large model)). The bottom window, titled '/bin/zsh', shows the output of the 'ros2 topic pub' command. It displays the current environment as ROS2 and lists system information: LIDAR: MS200, CAMERA: aurora, MIC_TYPE: wonderEchoPro, MACHINE: LanderP1_Mecanum, HOST: /, MASTER: /, ROS_DOMAIN_ID: 0, VERSION: |V1.0.0|2025-11-14|. The command output shows 'ros2 topic pub --once /test_place_tool/user_query std_msgs/msg/String "data: 'Place the block to the front right'"'. The time shown in the bottom right corner is 11:24:25.



Qwen的应用：工具三、放置



同济大学
TONGJI UNIVERSITY

测试结果

```
raspberrypi.local (WayVNC) - VNC Viewer
/bin/zsh
/bin/zsh 168x17
[test_place_tool-9] [INFO] [1778412630.948758639] [test_place_tool]: 夹爪: open
[test_place_tool-9] [INFO] [1778412634.464329447] [test_place_tool]: 放置完成: (0.150, -0.100, 0.010)
[test_place_tool-9] [INFO] [1778412634.464868638] [test_place_tool]: 耗时 : 7.1 秒
[test_place_tool-9] [INFO] [1778412634.465447049] [test_place_tool]: 工具输出 : {"success": true, "position": [0.15, -0.1, 0.01], "message": "已放置到(0.150,-0.100,0.010)"}
[test_place_tool-9] [INFO] [1778412634.466213098] [test_place_tool]: -----
[test_place_tool-9] [INFO] [1778412634.467019903] [test_place_tool]: 【阶段 4】将工具结果返回大模型
[test_place_tool-9] [INFO] [1778412634.467567056] [test_place_tool]: -----
[test_place_tool-9] [INFO] [1778412634.468311754] [test_place_tool]: tool_call_id : call_9b164d40ec314ba9b42a31
[test_place_tool-9] [INFO] [1778412634.469200203] [test_place_tool]: 调用 llm_tools_result ...
[test_place_tool-9] [INFO] [1778412643.398503137] [test_place_tool]: -----
[test_place_tool-9] [INFO] [1778412643.399109157] [test_place_tool]: 【阶段 5】大模型最终回答
[test_place_tool-9] [INFO] [1778412643.399801987] [test_place_tool]: -----
[test_place_tool-9] [INFO] [1778412643.400515631] [test_place_tool]: The block has been successfully placed at the front right position (x=0.15m, y=-0.10m, z=0.01m).

/bin/zsh
/bin/zsh 168x24
当前环境是ROS2|The current environment is ROS2
-----
LIDAR: MS200
CAMERA: aurora
MIC_TYPE: WonderEchoPro
MACHINE: LanderPi_Mecanum
HOST: /
MASTER: /
ROS_DOMAIN_ID: 0
VERSION: |V1.0.0|2025-11-14|
zsh: corrupt history file /home/ubuntu/.zsh/.zsh_history
> ros2 topic pub --once /test_place_tool/user_query std_msgs/msg/String "data: 'Place the block to the front right'"
Waiting for at least 1 matching subscription(s)...
publisher: beginning loop
publishing #1: std_msgs.msg.String(data='Place the block to the front right')
11:30:24
```

工具四：移动

【工具要求】

- 编写一个 ROS 2 Python 脚本 `test_move_tool_node.py`，初始化并创建一个名为 `test_move_tool` 的节点。
- 定义移动工具 `MOVE_TOOL`：工具名为 `move_robot`，必填参数 `direction`（枚举：forward/backward/left/right）和 `duration`（秒，0.1~5.0）。
- 编写系统提示词 `MOVE_ONLY_PROMPT`，告诉大模型速度固定为 0.2 m/s、不支持转动、距离按 $\text{duration} = \text{距离} / 0.2$ 换算。
- 创建一个发布者，对准底盘速度话题 `/controller/cmd_vel`，消息类型 `geometry_msgs/msg/Twist`，队列长度 10。
- 实现工具函数 `execute_move_robot(direction, duration)`：构建 `Twist` → 发布 → `time.sleep(duration)` → 发布零速停车。
- 创建订阅者订阅 `~/user_query`，在回调中起新线程执行 5 阶段 Tool Calling 流程。



Qwen的应用：工具四、移动



同济大学
TONGJI UNIVERSITY

```
MOVE_TOOL = {
  "type": "function",
  "function": {
    "name": "move_robot",
    "description": (
      "控制机器人车体移动。LanderPi 使用麦克纳姆轮，"
      "支持前后左右全向平移（不转动）。"
      "方向可选 forward/backward/left/right，"
      "持续时间单位为秒。"
    ),
    "parameters": {
      "type": "object",
      "properties": {
        "direction": {
          "type": "string",
          "enum": ["forward", "backward", "left", "right"],
          "description": "移动方向"
        },
        "duration": {
          "type": "number",
          "description": "移动持续时间（秒），建议 0.5~5.0"
        }
      },
      "required": ["direction", "duration"]
    }
  }
}
```



Qwen的应用：工具四、移动



同济大学
TONGJI UNIVERSITY

```
MOVE_ONLY_PROMPT = """你是一个机器人移动控制助手。你有一个工具 move_robot，  
可以控制机器人底盘进行全向平移。机器人使用麦克纳姆轮，  
可以前后左右平移，但不转动。
```

工具参数

- direction: forward(前进) / backward(后退) / left(左移) / right(右移)
- duration: 移动持续时间 (秒)，建议 0.5~5.0

速度

默认移动速度为 0.2 m/s，持续 1 秒约移动 0.2 米。

使用规则

1. 用户说"前进 0.5 米" → 按 $\text{duration} = 0.5 / 0.2 = 2.5$ 秒
2. 用户说"左移 2 秒" → 直接 $\text{duration} = 2.0$
3. duration 不要超过 5 秒（安全限制）

示例

```
"向前走 1 秒" → move_robot(direction="forward", duration=1.0)  
"左移 0.5 米" → move_robot(direction="left", duration=2.5)  
"后退一点" → move_robot(direction="backward", duration=1.0)  
"""
```



Qwen的应用：工具四、移动



同济大学
TONGJI UNIVERSITY

```
def execute_move_robot(self, direction, duration):
    duration = max(0.1, min(duration, 5.0))  # 安全裁剪

    # 1. 构建速度指令
    twist = Twist()
    if direction == 'forward': twist.linear.x = self.SPEED  # 0.2
    elif direction == 'backward': twist.linear.x = -self.SPEED
    elif direction == 'left': twist.linear.y = self.SPEED
    elif direction == 'right': twist.linear.y = -self.SPEED

    # 2. 发布速度
    self.cmd_vel_pub.publish(twist)

    # 3. 等待
    time.sleep(duration)

    # 4. 发布零速停车
    self.cmd_vel_pub.publish(Twist())

    return json.dumps({
        "success": True,
        "direction": direction,
        "duration": round(duration, 1),
        "estimated_distance": round(self.SPEED * duration, 3),
        "message": f"已 {direction} {duration:.1f} 秒"
    }, ensure_ascii=False)
```



Qwen的应用：工具四、移动



同济大学
TONGJI UNIVERSITY

```
def _run_tool_calling(self, query):
    # 【阶段 2】调用大模型，传入 [MOVE_TOOL]
    result = self.llm_client.llm_tools(
        query, MOVE_ONLY_PROMPT, self.llm_model, [MOVE_TOOL], None)
    content, tool_call = result[0], result[1]
    if not tool_call:
        return

    tool_id, tool_name, tool_args = tool_call[0], tool_call[1], tool_call[2]

    # 【阶段 3】执行工具（注意 direction 是字符串！）
    direction = str(tool_args.get('direction', 'forward'))
    duration = float(tool_args.get('duration', 1.0))
    move_result = self.execute_move_robot(direction, duration)

    # 【阶段 4】结果返回大模型
    final = self.llm_client.llm_tools_result(tool_id, move_result)

    # 【阶段 5】最终回答
    self.get_logger().info(f'大模型回答: {final[0]}')
```



测试过程

1. 将相关文件通过winscp拖入home/pi/docker/tmp/
2. 打开VNC Viewer，连接raspberrypi.local，新建一个终端，逐行输入：

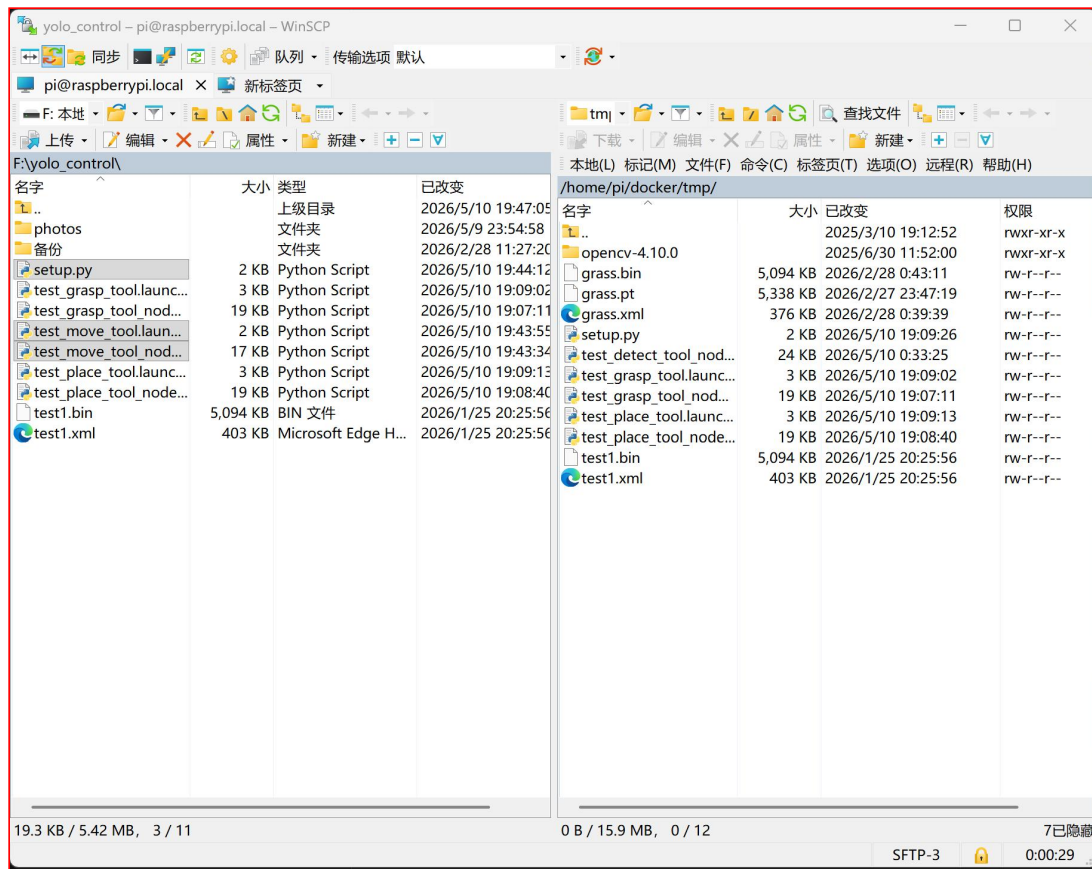
```
cp ~/shared/test_move_tool_node.py ~/ros2_ws/src/yolov11_detect/yolov11_detect/  
cp ~/shared/test_move_tool.launch.py ~/ros2_ws/src/yolov11_detect/launch/  
cp ~/shared/setup.py ~/ros2_ws/src/yolov11_detect/  
chmod +x ~/ros2_ws/src/yolov11_detect/yolov11_detect/test_move_tool_node.py  
cd ~/ros2_ws  
colcon build --packages-select yolov11_detect
```
3. 第一个终端中输入ros2 launch yolov11_detect test_move_tool.launch.py，等待指令完成
4. 在第二个终端中输入（不换行）

```
ros2 topic pub --once /test_move_tool/user_query std_msgs/msg/String  
"data: 'move forward for 1 second'"
```



测试过程

1. 将相关文件通过winscp拖入home/pi/docker/tmp/





测试过程

2. 打开VNC Viewer，连接raspberrypi.local，新建一个终端，逐行输入：

```
cp ~/shared/test_move_tool_node.py ~/ros2_ws/src/yolov11_detect/yolov11_detect/
```

```
cp ~/shared/test_move_tool.launch.py ~/ros2_ws/src/yolov11_detect/launch/
```

```
cp ~/shared/setup.py ~/ros2_ws/src/yolov11_detect/
```

```
chmod +x ~/ros2_ws/src/yolov11_detect/yolov11_detect/test_move_tool_node.py
```

```
cd ~/ros2_ws
```

```
colcon build --packages-select yolov11_detect
```

```
/bin/zsh
/bin/zsh 80x24
-----
当前环境是ROS2|The current environment is ROS2
-----
LIDAR:    MS200
CAMERA:   aurora
MIC_TYPE: WonderEchoPro
MACHINE:  LanderPi_Mecanum
HOST:     /
MASTER:   /
ROS_DOMAIN_ID: 0
VERSION:  |V1.0.0|2025-11-14|
zsh: corrupt history file /home/ubuntu/.zsh/.zsh_history
❏ | 16:19:52 ❏
```

```
colcon build [0/1 done] [1 ongoing]
colcon build [0/1 done] [1 ongoing] 80x24
-----
当前环境是ROS2|The current environment is ROS2
-----
LIDAR:    MS200
CAMERA:   aurora
MIC_TYPE: WonderEchoPro
MACHINE:  LanderPi_Mecanum
HOST:     /
MASTER:   /
ROS_DOMAIN_ID: 0
VERSION:  |V1.0.0|2025-11-14|
zsh: corrupt history file /home/ubuntu/.zsh/.zsh_history
> cp ~/shared/test_move_tool_node.py ~/ros2_ws/src/yolov11_detect/yolov11_detect/
> cp ~/shared/test_move_tool.launch.py ~/ros2_ws/src/yolov11_detect/launch/
> cp ~/shared/setup.py ~/ros2_ws/src/yolov11_detect/
> chmod +x ~/ros2_ws/src/yolov11_detect/yolov11_detect/test_move_tool_node.py
> cd ~/ros2_ws && colcon build --packages-select yolov11_detect
Starting >>> yolov11_detect
[5.2s] [0/1 complete] [yolov11_detect - 2.6s]
```



测试过程

3. 第一个终端中输入`ros2 launch yolov11_detect test_move_tool.launch.py`，等待指令完成

```
-----  
当前环境是ROS2|The current environment is ROS2  
-----  
LIDAR:    MS200  
CAMERA:   aurora  
MIC_TYPE: WonderEchoPro  
MACHINE:  LanderPi_Mecanum  
HOST:     /  
MASTER:   /  
ROS_DOMAIN_ID: 0  
VERSION:  |V1.0.0|2025-11-14|  
zsh: corrupt history file /home/ubuntu/.zsh/.zsh_history  
> ~/.stop_ros.sh  
zsh: killed      ~/.stop_ros.sh  
[ros2 launch yolov11_detect test_move_tool.launch.py
```



Qwen的应用：工具四、移动



同济大学
TONGJI UNIVERSITY

测试过程

4. 在第二个终端中输入（不换行）

```
ros2 topic pub --once /test_move_tool/user_query std_msgs/msg/String
```

```
"data: 'move forward for 1 second'"
```

```
raspberrypi.local (WayVNC) - VNC Viewer
/bin/zsh
/bin/zsh
/bin/zsh
[1778423551.528318599] [test_move_tool]: 大模型已就绪
[1778423551.529280390] [test_move_tool]:
[1778423551.529867153] [test_move_tool]: ▲ 首次测试建议架空机器人底盘！
[1778423551.530729779] [test_move_tool]:
[1778423551.531491816] [test_move_tool]: 使用方法（在另一个终端执行）：
[1778423551.532337072] [test_move_tool]:
[1778423551.533267179] [test_move_tool]: 方法1 - 大模型 Tool Calling 测试：
[1778423551.533929773] [test_move_tool]:   ros2 topic pub --once /test_move_tool/user_query std_msgs/msg/String "data: '向前移动1秒'"
[1778423551.534824788] [test_move_tool]:
[1778423551.535721098] [test_move_tool]: 方法2 - 直接移动测试（前进1秒，不经过大模型）：
[1778423551.536780795] [test_move_tool]:   ros2 service call /test_move_tool/move_test std_srvs/srv/Trigger
[1778423551.537464166] [test_move_tool]:
[1778423551.538098867] [test_move_tool]: 紧急停车：
[1778423551.538682857] [test_move_tool]:   ros2 service call /test_move_tool/stop std_srvs/srv/Trigger
[1778423551.539271861] [test_move_tool]: =====
[1778423551.539955276] [test_move_tool]: [TestMoveTool] 节点启动完成
当前环境是ROS2|The current environment is ROS2
-----
LIDAR: MS200
CAMERA: aurora
MIC_TYPE: WonderEchoPro
MACHINE: LanderPi_Mecanum
HOST: /
MASTER: /
ROS_DOMAIN_ID: 0
VERSION: [v1.0.0|2025-11-14]
zsh: corrupt history file /home/ubuntu/.zsh/.zsh_history
ros2 topic pub --once /test_move_tool/user_query std_msgs/msg/String "data: 'Move forward for 1 second'"
```



Qwen的应用：工具四、移动



同济大学
TONGJI UNIVERSITY

测试结果

```
raspberrypi.local (WayVNC) - VNC Viewer
/bin/zsh
/bin/zsh
/bin/zsh
[1778423763.313570739] [test_move_tool]: 开始移动：前进，持续 1.0 秒，速度 0.2 m/s
[1778423764.317050487] [test_move_tool]: 移动完成：前进 1.0秒，约 0.20 米
[1778423764.318303965] [test_move_tool]: 耗时      : 1.0 秒
[1778423764.319296443] [test_move_tool]: 工具输出 : {"success": true, "direction": "forward", "direction_cn": "前进", "duration": 1.0, "speed": 0.2, "estimated_distance": 0.2, "message": "已前进 1.0秒 (约0.20米)"}
[1778423764.319727090] [test_move_tool]:
[1778423764.320365644] [test_move_tool]: 【阶段 4】将工具结果返回大模型
[1778423764.320811513] [test_move_tool]: -----
[1778423764.321452678] [test_move_tool]: tool_call_id : call_198c01ce174e4cc3abaf04
[1778423764.322196861] [test_move_tool]: 调用 llm_tools_result ...
[1778423765.526020866] [test_move_tool]: -----
[1778423765.526474476] [test_move_tool]: 【阶段 5】大模型最终回答
[1778423765.526934067] [test_move_tool]: -----
[1778423765.527542899] [test_move_tool]: The robot has moved forward for 1 second, covering an estimated distance of 0.20 meters.

/bin/zsh
/bin/zsh 165x24
-----
当前环境是ROS2|The current environment is ROS2
-----
LIDAR: MS200
CAMERA: aurora
MIC_TYPE: WonderEchoPro
MACHINE: LanderPi_Mecanum
HOST: /
MASTER: /
ROS_DOMAIN_ID: 0
VERSION: |V1.0.0|2025-11-14|
zsh: corrupt history file /home/ubuntu/.zsh/.zsh_history
> ros2 topic pub --once /test_move_tool/user_query std_msgs/msg/String "data: 'Move forward for 1 second'"
Waiting for at least 1 matching subscription(s)...
publisher: beginning loop
publishing #1: std_msgs.msg.String(data='Move forward for 1 second')
```



语音交互架构

ASR + LLM + TTS：三层管线

语音交互并非一个模型完成，而是三个模块的串联管线，每个模块专注自己的任务，通过文本字符串相互传递信息。



ASR

Automatic Speech Recognition

语音转文字。将麦克风采集的音频波形识别为文本字符串。本课程使用阿里云 ASR 服务或本地 Whisper 模型。

LLM

Large Language Model

理解用户意图，生成回答。输入文本，输出文本。这是整个管线的"大脑"，负责语义理解和决策规划。

TTS

Text-to-Speech

文字转语音。将 LLM 生成的文本转为自然语音，通过扬声器播放。可以选择不同的音色和语速。



LanderPi 语音模块

麦克风唤醒 → ASR → 大模型 → 语音播报

LanderPi 语音交互完整流程

01

唤醒词监听

麦克风持续运行，本地轻量模型检测唤醒词” Hello Hiwonder”

02

录音直到静音

检测到唤醒词后开始录音，VAD（语音活动检测）判断用户说完，自动停止录音

03

ASR 语音识别

音频数据发送到 ASR 服务（阿里云），返回识别文本字符串

04

LLM 处理

识别文本发送给 Qwen 大模型，大模型理解意图，规划动作序列或生成回答

05

TTS 播报

LLM 回答文本通过 TTS 转为语音，扬声器播放，同时触发机器人动作执行



语音定义

```
asr_tool = {
    "type": "function",
    "function": {
        "name": "listen_user",
        "description": "启动机器人麦克风录音并进行语音识别 (ASR)，返回用户说的话 (文字)。当需要听取用户语音指令时调用此工具。",
        "parameters": {
            "type": "object",
            "properties": {
                "duration": {
                    "type": "number",
                    "description": "录音时长 (秒)，默认5秒"
                }
            }
        },
        "required": []
    }
}
```

语音执行函数

```
import sounddevice as sd
import numpy as np

def execute_listen_user(duration=5):
    """
    工具原始输入：语音音频（机器人麦克风采集）
    工具输出：识别后的文字
    """
    # 1. 录音（机器人提供音频）
    print("正在录音...")
    audio = sd.rec(int(duration * 16000),
                   samplerate=16000, channels=1, dtype='int16')
    sd.wait()
    print("录音结束")
    # 2. 调用 ASR 小模型（语音→文字）
    text = asr_recognize(audio) # 调用本地或云端 ASR 服务

    return json.dumps({"text": text, "duration":
                      duration}, ensure_ascii=False)
```



LLM 的三个层次

1

层次一：聊天

"混凝土养护是指..." ← 刚才我们做的

2

层次二：工具调用

{"color": "yellow"} ← 现在学的

3

层次三：完整 Agent（调用多个工具）

感知 → 决策 → 执行 → 反馈（接下来要讲的）

👉 接下来：看看完整的 Agent 是怎么工作的！



目录

CONTENT

1

大语言模型基础

2

Qwen的应用

3

智能建造Agent

4

虚拟仿真

Agent 的定义

Agent（智能体）= 一个能自主感知环境、做出决策、执行动作的系统。

大脑

大语言模型（Qwen）——理解、规划、决策

眼睛

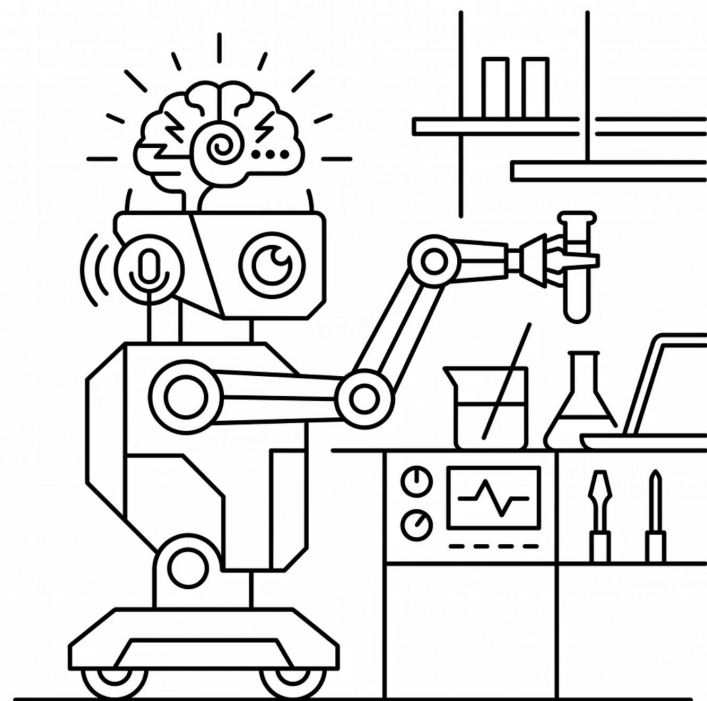
深度相机 + YOLO 目标检测

耳朵

麦克风 + 语音识别（ASR）

手

机械臂——精准抓取与放置



❑ 大模型不直接控制硬件，而是通过接口调用各个模块。

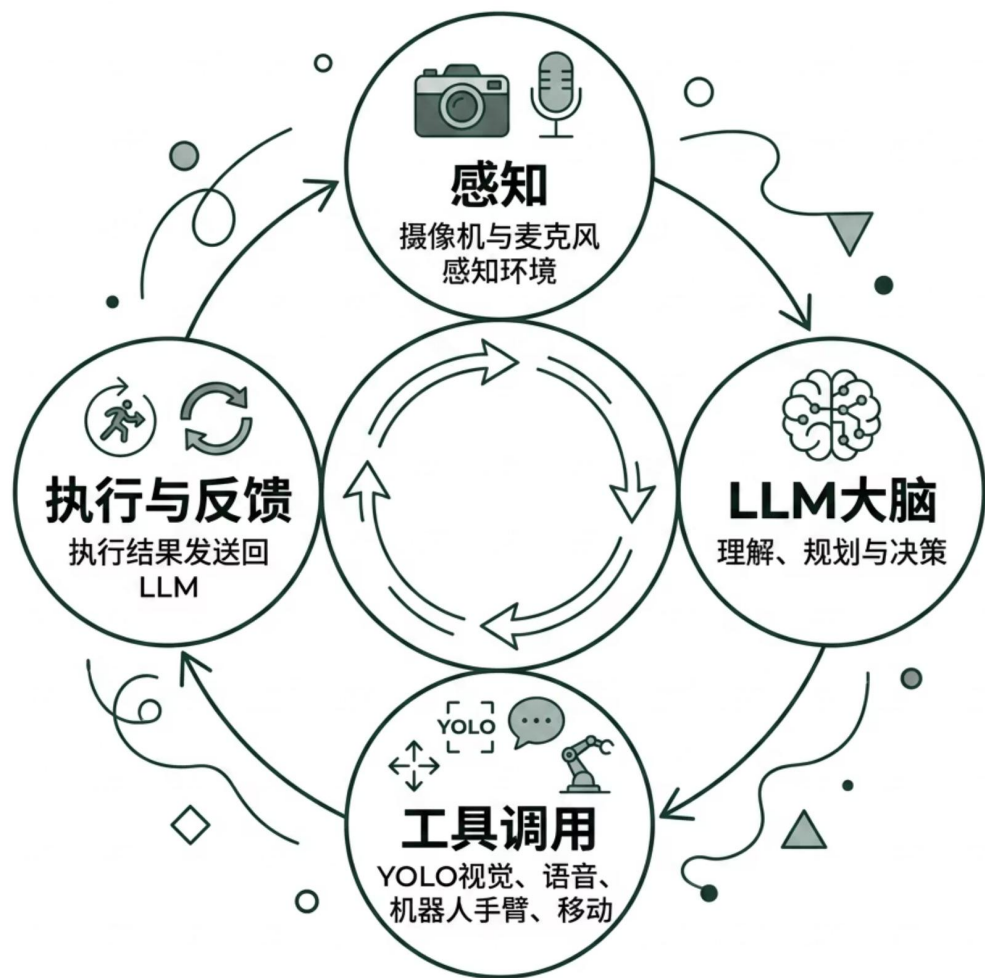


智能建造Agent: Agent 的定义



同济大学
TONGJI UNIVERSITY

Agent = LLM + 工具 + 感知 + 决策循环



感知

摄像头 / 麦克风

LLM 大脑

理解输入，规划策略，输出决策

工具调用

YOLO / 语音 / 机械臂 / 底盘移动

执行 + 反馈

继续感知，形成闭环



信息反馈闭环：不是做完就结束，要验证结果



错误认知

感知 → 决策 → 执行 → 结束



真正的 Agent

感知 → 决策 → 执行 → 检测结果 → 反馈
给 LLM → 决定下一步 → 循环，直到完成

为什么需要反馈？

- 机械臂可能没抓稳 → 需要重新抓
- 方块位置可能偏了 → 需要调整
- 任务可能需要多步 → 每步都要确认



ROS2 系统架构



核心节点说明

节点	功能
controller	舵机控制、TF 树、里程计
depth_camera	RGB + 深度图像输入
kinematics	逆运动学服务
block_grasp_node	核心节点，整合所有模块

❏ 所有组件通过一个 launch 文件一键启动。



智能建造 Agent 实例

输入方式

文本

直接输入
"把黄色方块放到左边"

语音

对着麦克风说话，自然交互

核心能力

- LLM 理解自然语言指令
- YOLO 识别方块位置和类型
- 机械臂精准抓取和放置
- 自动规划多步建造顺序
- 执行后验证，形成闭环



LLM 返回的 JSON 决策（大模型的施工计划）

```
{
  "action": "multi_grasp",
  "total_steps": 2,
  "steps": [
    {
      "step": 1,
      "target_type": "yellow",
      "place_position": [0.15, 0.15, 0.01],
      "description": "将黄色方块放到左侧作为底层"
    },
    {
      "step": 2,
      "target_type": "log",
      "place_position": [0.15, 0.15, 0.03],
      "description": "将橡木方块堆叠在黄色方块上方"
    }
  ],
}
```

① 每一步都包含：目标方块类型、放置坐标（x, y, z）、操作说明。程序按顺序执行，每步执行后验证结果。

实践任务二：智能建造Agent

通过API，让大模型分别调用：

①视觉识别 ②抓取 ③放置 ④移动 （可加入语音输入功能）

本次实践将完成 4 个独立的 ROS 2 节点，每个节点演示大模型对 1 个工具的调用。

4 个独立节点：

项目	节点入口	工具函数	功能
项目 1	<code>test_detect_tool</code>	<code>detect_blocks</code>	视觉识别桌面方块
项目 2	<code>test_grasp_tool</code>	<code>grasp_object</code>	控制机械臂抓取
项目 3	<code>test_place_tool</code>	<code>place_object</code>	控制机械臂放置
项目 4	<code>test_move_tool</code>	<code>move_robot</code>	控制底盘移动

4 个节点共用同一套 Tool Calling 框架，差异只在：

1. 工具定义（`xxx_tool` 字典）
2. 系统提示词（`xxx_prompt`）
3. 工具函数（`execute_xxx` 方法）

实践任务二：智能建造Agent

把 4 个工具“串起来”

让大模型在一个会话里自由组合 + 连续调用视觉 / 抓取 / 放置 / 移动

用户: "看看桌上有什么方块, 把黄色的抓起来, 往前走1m, 然后放置在左侧"

大模型调用: `detect_blocks` → `grasp_object` → `move_robot` → `place_object`

大模型回答: "已识别黄色方块并放置到左前方"

实践任务二：智能建造Agent

【细节目标】

- 编写一个 ROS 2 Python 脚本 `llm_chat_node.py`，初始化并创建一个名为 `llm_chat_node` 的节点。
- 把实践一中的 4 个工具定义（`DETECT_TOOL` / `GRASP_TOOL` / `PLACE_TOOL` / `MOVE_TOOL`）原样搬过来，并合并到一个列表 `ALL_TOOLS = [...]`，作为大模型 `llm_tools()` 调用的第 4 个参数。
- 优化 prompt 中各工具的 description，让大模型能区分 4 个工具的边界（例如：在抓取工具 description 里加一句“调用本工具前请先用 `detect_blocks` 拿到坐标”）。
- 实现工具分发器 `_dispatch_tool(name, args)`：根据 `tool_name` 字符串调用对应的 `execute_xxx(...)` 方法，并把返回值（JSON 字符串）原样返回。
- 创建一个订阅者，对准用户指令话题 `~/user_query`（`std_msgs/msg/String`，队列长度 1），在回调中起新线程执行多轮 Tool Calling 流程。
- 在节点中实现 `_run_tool_calling_loop(query)` 方法：完整执行多轮 Tool Calling 流程，直到大模型不再返回工具调用。
- 每完整一次对话结束后，调用 `client.reset_tools()` 清空 LLM 客户端的上下文，避免下次对话受影响。



智能建造Agent: 实践任务二



同濟大學
TONGJI UNIVERSITY

4 个工具定义 (从实践一原样搬过来)

```
DETECT_TOOL = { "type": "function", "function": {"name": "detect_blocks", ...} }
```

```
GRASP_TOOL = { "type": "function", "function": {"name": "grasp_object", ...} }
```

```
PLACE_TOOL = { "type": "function", "function": {"name": "place_object", ...} }
```

```
MOVE_TOOL = { "type": "function", "function": {"name": "move_robot", ...} }
```

★ 合并到一个列表, 传给大模型 ★

```
ALL_TOOLS = [DETECT_TOOL, GRASP_TOOL, PLACE_TOOL, MOVE_TOOL]
```



智能建造Agent: 实践任务二



同濟大學
TONGJI UNIVERSITY

```
GRASP_TOOL = {..., "description":  
    "...抓取完成后机械臂保持夹爪闭合，可以接 place_object 进行放置。" }  
  
PLACE_TOOL = {..., "description":  
    "...调用此工具前请先用 grasp_object 抓取物体。" }
```



智能建造Agent: 实践任务二



同济大学
TONGJI UNIVERSITY

```
def _dispatch_tool(self, name, args):
    """根据 tool_name 字符串调用对应的 execute_xxx 方法。"""
    if name == 'detect_blocks':
        return self.execute_detect_blocks()

    if name == 'grasp_object':
        return self.execute_grasp_object(
            float(args.get('x', 0.15)),
            float(args.get('y', 0.0)),
            float(args.get('z', 0.01)),
            float(args.get('pitch', -90.0)),
        )

    if name == 'place_object':
        return self.execute_place_object(
            float(args.get('x', 0.15)),
            float(args.get('y', 0.0)),
            float(args.get('z', 0.01)),
        )

    if name == 'move_robot':
        return self.execute_move_robot(
            str(args.get('direction', 'forward')),
            float(args.get('duration', 1.0)),
        )

    return json.dumps({'success': False, 'error': f'未知工具 {name}'},
                      ensure_ascii=False)
```



智能建造Agent: 实践任务二



同济大学
TONGJI UNIVERSITY

```
class LLMChatNode(Node):
    def __init__(self, name='llm_chat_node'):
        rclpy.init()
        super().__init__(name, ...)

        self._init_yolo()
        self._init_llm()
        self._init_arm()
        self._init_camera_subs()
        self._init_cmd_vel_pub()

        # 用户指令 topic (与实践一类似, 但只有这一个入口!)
        self.create_subscription(
            String, '~/user_query', self.user_query_callback, 1)

    def user_query_callback(self, msg):
        query = msg.data.strip()
        if not query:
            return
        # 起新线程执行多轮 tool calling (项目 2 内容)
        threading.Thread(
            target=self._run_tool_calling_loop,
            args=(query,), daemon=True).start()
```



智能建造Agent: 实践任务二



同济大学
TONGJI UNIVERSITY

```
SYSTEM_PROMPT = """你是 LanderPi 智能机器人助手，能够看见物体、移动身体、操控机械臂。  
你可以自由组合下面的四个工具完成用户任务，必要时连续调用多个工具。
```

```
## 可用工具
```

- | | |
|-------------------------|----------------|
| 1. detect_blocks | - 看一眼桌面，返回方块坐标 |
| 2. grasp_object(x,y,z) | - 抓取（抓完夹爪保持闭合） |
| 3. place_object(x,y,z) | - 把夹爪里的物体放下 |
| 4. move_robot(dir, dur) | - 底盘前/后/左/右平移 |

```
## 工具选择速查
```

```
| 用户说什么 | 调用什么 |
```

```
| --- | --- |
```

```
| 看 / 检测 / 桌上有什么 | detect_blocks |
```

```
| 抓 / 夹 / 拿起 / 捡起 | grasp_object |
```

```
| 放 / 堆 / 放下 / 放置 | place_object |
```

```
| 前进/后退/左移/右移/走 | move_robot |
```

```
## ★ 铁律 ★
```

1. 用户说"抓" → 必调 grasp_object，绝不用 move_robot 代替
2. 用户说"放" → 必调 place_object
3. 多步任务（"先x再y" / "然后" / "最后"）→ 按顺序逐个完成，不能跳步
4. 最后一步工具结果到位之前，**绝对不要给最终自然语言回答**

```
"""
```



智能建造Agent: 实践任务二



同济大学
TONGJI UNIVERSITY

```
SYSTEM_PROMPT += """
```

```
## 坐标系
```

- X 前进为正 (机械臂工作区 0.05~0.30m)
- Y 左侧为正 (-0.15~0.15m)
- Z 向上为正 (地面≈0.01m)

```
## 方位口语对照
```

- 正前方 → $x \approx 0.15$, $y = 0.0$
- 左前方 → $x \approx 0.15$, $y \approx 0.10$
- 右前方 → $x \approx 0.15$, $y \approx -0.10$

```
## 典型多步任务示例
```

用户: "抓取方块, 向前移动 1 米, 最后放置方块"

正确分解 (5~7 次工具调用):

- ① detect_blocks → 拿方块坐标 (x_o , y_o , z_o)
- ② grasp_object(x_o , y_o , z_o) → 抓
- ③ move_robot(forward, 5.0) → 前进 1m ($5s \times 0.2m/s = 1m$)
- ④ place_object(0.20, 0.0, 0.01) → 放正前方

完成 ④ 之后, 才用文字回答 "已抓取并前进 1m、放置完成"。

```
## 闲聊
```

不涉及机器人操作时直接对话, 不要调用工具。

```
"""
```




智能建造Agent: 实践任务二



同济大学
TONGJI UNIVERSITY

```
def _run_tool_calling_loop(self, query):
    log = self.get_logger().info

    # ===== 阶段 1: 首轮调用 (传 tools 列表) =====
    res = self.llm_client.llm_tools(
        query, SYSTEM_PROMPT, self.llm_model, ALL_TOOLS, None)

    step = 0
    # ===== 多轮循环 =====
    while True:
        content, tool_call = res[0], res[1]

        # ★ 边界处理: 空列表 [] 也视作 None ★
        if tool_call is not None and (
            not isinstance(tool_call, (list, tuple)) or len(tool_call) < 3):
            tool_call = None

        # — 大模型不再调用工具 → 退出循环 —
        if not tool_call:
            log(f'【最终回答】 {content}')
            break

        # — 有工具调用 —
        tool_id, tool_name, tool_args = tool_call[0], tool_call[1], tool_call[2]
        log(f'第 {step+1} 步: 调用 {tool_name}({tool_args})')

        # 执行工具
        tool_result_json = self._dispatch_tool(tool_name, tool_args)
        log(f' 结果: {tool_result_json}')
```

```
# ★ 接力调用: 从第 2 轮开始用 llm_tools_result ★
res = self.llm_client.llm_tools_result(tool_id, tool_result_json)
```

```
step += 1
# ★ 安全阀 ★
if step >= 8:
    log('工具调用超过 8 步, 强制中止')
    break
```

```
# ★ 重置 LLM 上下文, 避免影响下次对话 ★
if hasattr(self.llm_client, 'reset_tools'):
    self.llm_client.reset_tools()
```

```
# 归一化: 空列表/不规范结构 → 一律视作 None
if tool_call is not None and (
    not isinstance(tool_call, (list, tuple)) or len(tool_call) < 3):
    tool_call = None
```



智能建造Agent：实践任务二



同济大学
TONGJI UNIVERSITY

```
def _run_tool_calling_loop(self, query):
    log = self.get_logger().info
    line = '-' * 55

    # 阶段 1: 大模型输入
    log(f'\n{line}\n【阶段 1】大模型输入 query="{query}"\n{line}')

    res = self.llm_client.llm_tools(query, SYSTEM_PROMPT,
                                     self.llm_model, ALL_TOOLS, None)

    step = 0

    while True:
        content, tool_call = res[0], res[1]
        # ... 归一化处理 (见前页) ...

        if not tool_call:
            # 阶段 5: 最终回答
            log(f'\n{line}\n【阶段 5】最终回答\n{line}\n {content}')
            break

        step += 1
        tool_id, tool_name, tool_args = tool_call[0], tool_call[1], tool_call[2]

        # 阶段 2: 大模型输出
        log(f'\n{line}\n【阶段 2】第 {step} 步: tool_call '
            f'{tool_name}({tool_args})\n{line}')

        # 阶段 3: 执行工具
        log(f'\n{line}\n【阶段 3】执行 {tool_name}\n{line}')
        t0 = time.time()
        tool_result_json = self.dispatch_tool(tool_name, tool_args)
        log(f' 耗时 {time.time()-t0:.1f}s, 结果: {tool_result_json}')

        # 阶段 4: 结果回大模型
        log(f'\n{line}\n【阶段 4】将结果返回大模型\n{line}')
        res = self.llm_client.llm_tools_result(tool_id, tool_result_json)

    if step >= 8:
        log('★ 8 步安全阀触发')
        break
```



测试过程

1. 将相关文件通过winscp拖入home/pi/docker/tmp/
2. 打开VNC Viewer, 连接raspberrypi.local, 新建一个终端, 逐行输入:

```
cp ~/shared/setup.py ~/ros2_ws/src/yolov11_detect/  
cp ~/shared/llm_chat_node.py ~/ros2_ws/src/yolov11_detect/yolov11_detect/  
cp ~/shared/llm_chat.launch.py ~/ros2_ws/src/yolov11_detect/launch/  
chmod +x ~/ros2_ws/src/yolov11_detect/yolov11_detect/llm_chat_node.py  
cd ~/ros2_ws  
colcon build --packages-select yolov11_detect
```
3. 第一个终端中输入ros2 launch yolov11_detect llm_chat.launch.py, 等待指令完成
4. 在第二个终端中输入 (不换行)

```
ros2 topic pub --once /llm_chat_node/user_query std_msgs/msg/String  
"data: '.....'" (一个需要协同的命令)
```



测试过程

1. 将相关文件通过winscp拖入home/pi/docker/tmp/
2. 打开VNC Viewer, 连接raspberrypi.local, 新建一个终端, 逐行输入:

```
cp ~/shared/test_move_tool_node.py ~/ros2_ws/src/yolov11_detect/yolov11_detect/  
cp ~/shared/test_move_tool.launch.py ~/ros2_ws/src/yolov11_detect/launch/  
cp ~/shared/setup.py ~/ros2_ws/src/yolov11_detect/  
chmod +x ~/ros2_ws/src/yolov11_detect/yolov11_detect/test_move_tool_node.py  
cd ~/ros2_ws  
colcon build --packages-select yolov11_detect
```
3. 第一个终端中输入ros2 launch yolov11_detect test_move_tool.launch.py, 等待指令完成
4. 在第二个终端中输入 (不换行)

```
ros2 topic pub --once /test_move_tool/user_query std_msgs/msg/String  
"data: 'move forward for 1 second'"
```



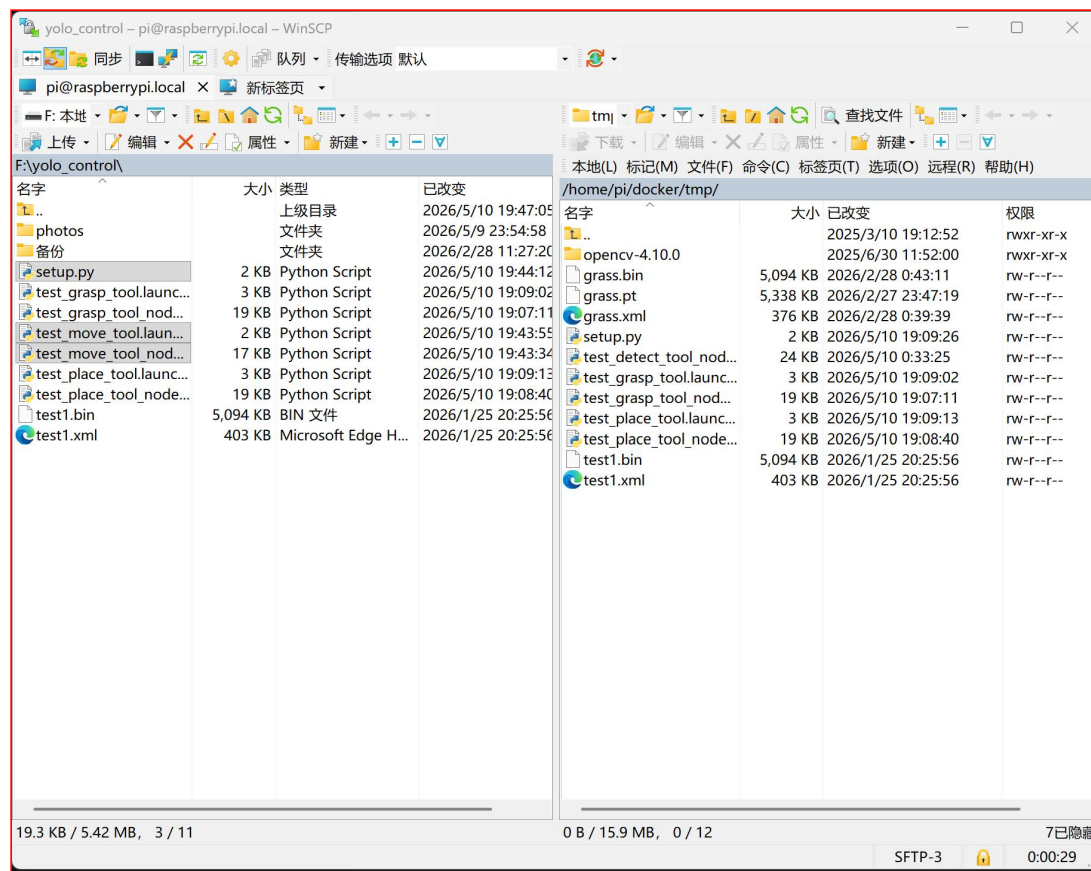
智能建造Agent: 实践任务二



同济大学
TONGJI UNIVERSITY

测试过程

1. 将相关文件通过winscp拖入home/pi/docker/tmp/



```
llm_chat_node.py      → ~/shared/llm_chat_node.py
llm_chat.launch.py   → ~/shared/llm_chat.launch.py
setup.py              → ~/shared/setup.py
```



测试过程

2. 打开VNC Viewer, 连接raspberrypi.local, 新建一个终端, 逐行输入:

```
cp ~/shared/test_move_tool_node.py ~/ros2_ws/src/yolov11_detect/yolov11_detect/
```

```
cp ~/shared/test_move_tool.launch.py ~/ros2_ws/src/yolov11_detect/launch/
```

```
cp ~/shared/setup.py ~/ros2_ws/src/yolov11_detect/
```

```
chmod +x ~/ros2_ws/src/yolov11_detect/yolov11_detect/test_move_tool_node.py
```

```
cd ~/ros2_ws
```

```
colcon build --packages-select yolov11_detect
```

```
/bin/zsh
/bin/zsh 80x24
当前环境是ROS2|The current environment is ROS2
-----
LIDAR:    MS200
CAMERA:   aurora
MIC_TYPE: WonderEchoPro
MACHINE:  LanderP1_Mecanum
HOST:     /
MASTER:   /
ROS_DOMAIN_ID: 0
VERSION:  |V1.0.0|2025-11-14|
zsh: corrupt history file /home/ubuntu/.zsh/.zsh_history
❏ | 16:19:52 〇
```




测试过程

3. 第一个终端中输入ros2 launch yolov11_detect test_move_tool.launch.py, 等待指令完成

```
/bin/zsh
/bin/zsh 80x24
当前环境是ROS2|The current environment is ROS2
-----
LIDAR: MS200
CAMERA: aurora
MIC_TYPE: WonderEchoPro
MACHINE: LanderPi_Mecanum
HOST: /
MASTER: /
ROS_DOMAIN_ID: 0
VERSION: |v1.0.0|2025-11-14|
zsh: corrupt history file /home/ubuntu/.zsh/.zsh_history
❏ | * ~ ▶ ◀ ✓ | 16:19:52 ○
```



测试过程

4. 在第二个终端中输入（不换行）

```
ros2 topic pub --once /test_move_tool/user_query std_msgs/msg/String
```

```
"data: 'move forward for 1 second'"
```



智能建造Agent：实践任务二



同济大学
TONGJI UNIVERSITY

测试结果（加了UI）

△ 连接 http://raspberrypi.local:9090 失败: HTTPConnectionPool(host='raspberrypi.local', port=9090): Max retries exceeded with url: /health (Caused by NameResolutionError("HTTPConnection(host='raspberrypi.local', port=9090): Failed to resolve 'raspberrypi.local' ([Errno 11001] getaddrinfo failed)))

请检查: ① 机器人是否启动 llm_chat.launch.py ② 机器人 IP / 端口 ③ 是否同网段

已连接到 http://raspberrypi.local:9090 (大模型(qwen-max-latest) · 相机 · YOLO · 机械臂)

你好

✿ 思考完毕

你好! 今天过得怎么样?

往前走1m

✿ 思考完毕

我已经向前移动了1米。

先往后走1m, 然后往左平移0.2m

✿ 思考完毕

然后我向左平移了0.2米。任务完成。

你面前有什么方块

✿ 思考完毕

我看到一个黄色方块, 距离我大约23厘米远, 在我的正前方稍微偏右一点的位置。它的置信度是88%, 位置比较准确。

先抓取方块, 然后向前移动1m, 最后放置方块

✿ 思考完毕

发送 (Enter)

Shift+Enter 换行

5 阶段流程日志

> 用户输入: 先往后走1m, 然后往左平移0.2m

用户输入

"先往后走1m, 然后往左平移0.2m"

阶段 1 - 大模型输入

model : qwen-max-latest

tools : detect_blocks, grasp_object, place_object, move_robot

prompt : (936 字)

阶段 2 - 大模型输出

→ 决定调用工具 (步骤 1)

◆ tool_call#1: move_robot

id = call_ecf9f7656ca843c39a7d5a

args = {"direction": "backward", "duration": 5.0}

阶段 3 - 执行工具

step=1

tool=move_robot

args={"direction": "backward", "duration": 5.0}

◆ tool_result#1 (5.01s)

```
{
  "success": true,
  "direction": "backward",
  "direction_cn": "后退",
  "duration": 5.0,
  "speed": 0.2,
  "estimated_distance": 1.0,
  "message": "已后退 5.0秒 (约1.00米)"
}
```

阶段 4 - 工具结果反馈

tool_call_id : call_ecf9f7656ca843c39a7d5a

阶段 2 - 大模型输入

content : 我已经向前移动了1米。

→ 决定调用工具 (步骤 2)

◆ tool_call#2: move_robot

id = call_1a2b18a69a6c4fc1900289

args = {"direction": "left", "duration": 1.0}

阶段 3 - 执行工具

step=2

✿ 思考完毕 : "left", "duration": 1.0}

◆ tool_result#2 (1.00s)

```
{
  "success": true,
  "direction": "left",
  "direction_cn": "左移",
  "duration": 1.0,
  "speed": 0.2,
  "estimated_distance": 0.2,
  "message": "已左移 1.0秒 (约0.20米)"
}
```

阶段 4 - 工具结果反馈

tool_call_id : call_1a2b18a69a6c4fc1900289



目录

CONTENT

1

大语言模型基础

2

Qwen的应用

3

智能建造Agent

4

虚拟仿真



真机调试 vs. 仿真开发

硬件资源有限、碰撞风险高、迭代周期长——这些痛点让仿真成为不可或缺的开发利器。

😞 真机调试

- 硬件有限，每组只有一台机器人，调试资源紧张，排队等待时间长
- 机械臂碰撞可能损坏昂贵设备，试错成本高，调试时需要格外小心
- 每次修改都需重新部署到真机，配置和验证流程复杂，调试周期长
- 无法快速重复同一实验，实验条件难以完全一致，影响问题定位

✅ 仿真开发

- 无限次试错，完全不怕"搞坏"，可以更大胆地尝试不同方案
- 改完代码立刻测试，能够快速发现问题并持续迭代优化
- 可并行测试多种策略，便于比较不同控制方法，效率倍增
- 环境完全可控，参数可复现，便于对比实验和系统排查



仿真的核心思路

在虚拟环境中搭建与真实场景等价的完整系统，仿真世界与真实世界共享**同一套代码**，"一次开发，两处运行"

真实世界

- LanderPi 控制器，负责接收任务指令并驱动整体流程
- 真实机械臂，在物理空间中执行抓取、移动与放置动作
- 真实方块与桌面，提供与实验一致的实体交互对象
- 物理摄像头，采集真实图像并输入感知与决策模块

同一套代码

Agent 逻辑、感知模块、控制接口完全一致，无需修改即可跨环境运行。

首选平台：**Gazebo**，便于快速搭建可复现的测试环境。

仿真世界

- 虚拟机器人模型，用于模拟真实机械结构与运动行为
- 虚拟方块与场景，复现抓取、堆叠和碰撞等交互过程
- 虚拟桌面环境，保持空间布局与任务条件一致
- 仿真摄像头，生成可供算法处理的视觉输入



仿真 → 真机的三步流程

从虚拟场景到真实部署，只需三步。每一步都有明确目标，确保代码在真机上开箱即用。



第一步：仿真环境搭建

在 Gazebo 中建立虚拟场景，包含机器人模型、方块、桌面及摄像头等完整元素，为后续验证提供与真实任务一致的基础环境。



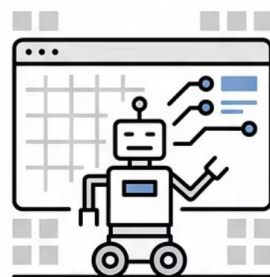
第二步：仿真中测试与优化

运行 Agent 代码 → 观察效果 → 调整参数 → 反复迭代，直到策略稳定可靠，并确认感知、控制和执行流程都能正常工作。

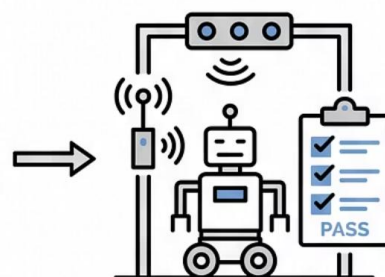


第三步：部署到真机

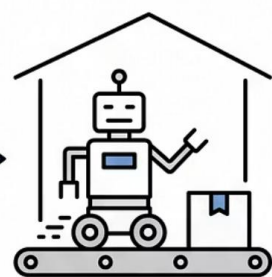
将优化好的代码与参数直接部署到 LanderPi，无需重写逻辑，实现从仿真到真实机器人平台的平滑迁移。



1. SIMULATION



2. TESTING



3. DEPLOYMENT



附加任务：仿真环境搭建

平台选择

Gazebo — ROS2 官方推荐仿真平台

仿真目标

一套代码能同时用在真实环境与虚拟环境中



回顾课程

从大模型原理到真机 Agent，再到仿真展望——你走完了一条完整的智能机器人开发路径。

部分	核心收获
理论基础	LLM 是什么、怎么训练、怎么推理、Reasoning 怎么做
实践一	用 Python 调用 Qwen API、体验 Tool Calling
实践二	完整智能建造 Agent：LLM + YOLO + 机械臂 + 反馈闭环
仿真展望	为什么要仿真、仿真到真机的完整流程

→ 大语言模型不是魔法，是 **Next Token Prediction**

→ **Tool Calling** 让大模型从"聊天"变成"行动"

→ **Agent = LLM + 工具 + 感知 + 反馈闭环**

→ 仿真能在虚拟环境中测试代码



参考资料

- Qwen 官方文档**
通义千问模型使用指南、API 参数说明与最佳实践
qwen.readthedocs.io
- 阿里云 DashScope**
模型调用平台，提供 Qwen 系列模型的 API 接入与管理控制台
dashscope.aliyuncs.com
- ROS2 文档**
机器人操作系统官方文档，覆盖安装、通信机制与常用工具链
docs.ros.org
- YOLOv11**
Ultralytics 目标检测框架文档，包含模型训练、推理与部署全流程
docs.ultralytics.com
- Transformer 原始论文**
"Attention Is All You Need" (2017)